

Sevak Documentation

Description	User guide and reference for Sevak, a keyboard-first desktop launcher for Windows, macOS and Linux.
Repository	https://github.com/ninad-k/Sevak
Copyright	Sevak is licensed under the Apache License 2.0.

Table of Contents

[Home](#)

[Getting started](#)

[Install](#)

[Quick start](#)

[User guide](#)

[Searching and launching](#)

[Keyboard shortcuts](#)

[Settings window](#)

[Themes](#)

[Features](#)

[Features overview](#)

[Applications](#)

[Calculator and conversions](#)

[Web search](#)

[Files and folders](#)

[Browser bookmarks](#)

[Clipboard history](#)

[Snippets](#)

[System commands](#)

[Shell commands](#)

[Selection actions](#)

[Script plugins](#)

[Reference](#)

[Configuration file](#)

[Command line](#)

[Files and data](#)

[Help](#)

[Troubleshooting](#)

[FAQ](#)

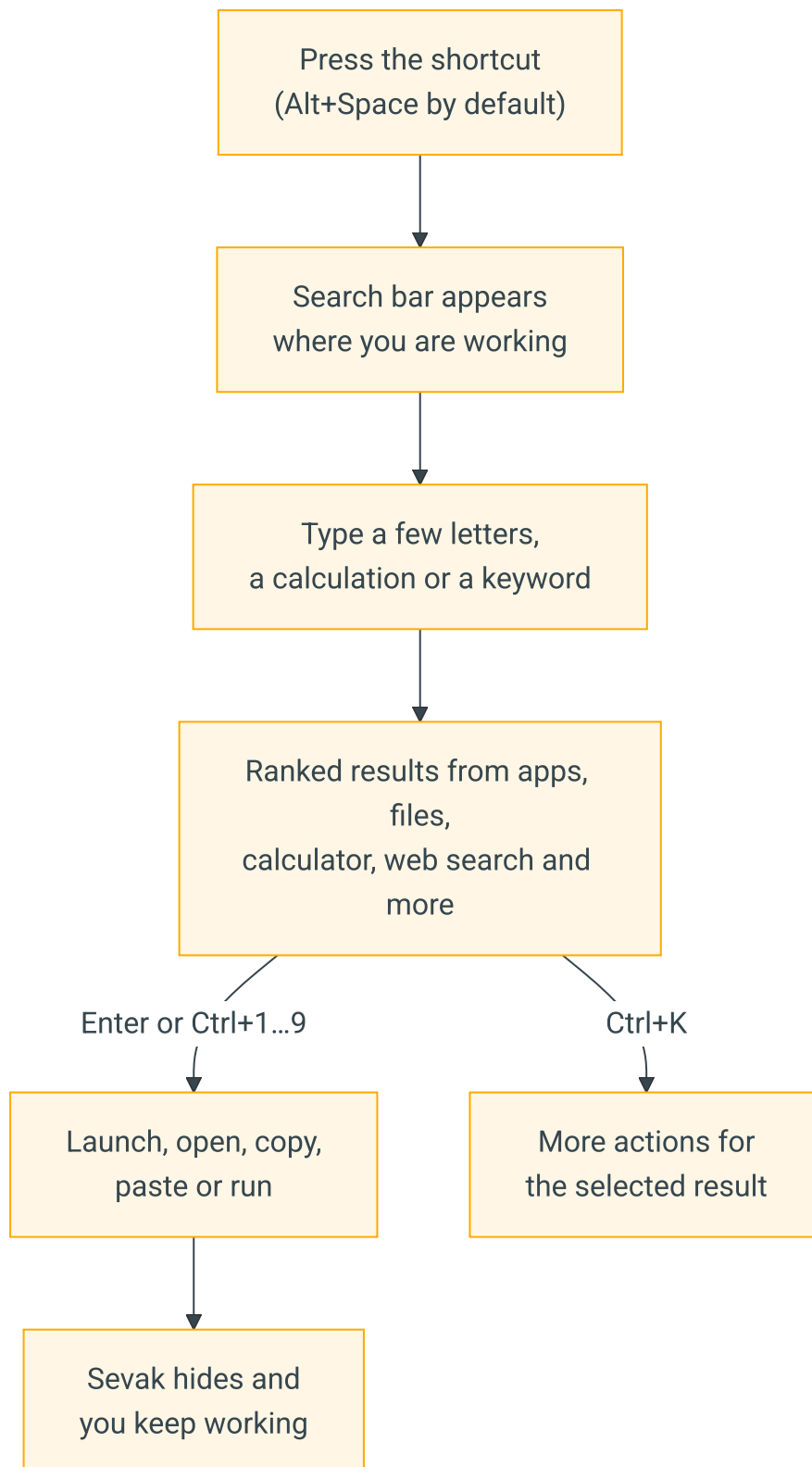
[Privacy](#)

Sevak documentation

Your desktop. At your service. Sevak is a keyboard-first launcher for Windows, macOS and Linux. Press a shortcut, type what you need, and press `Enter ↵` to open an app, find a file, calculate, convert units, search the web, paste a snippet or run a command, all without leaving the keyboard.



How it works in one picture



Where to go next

New to Sevak

Everyday use

Install it, then follow the [five-minute quick start](#).

[Searching and launching](#), the [keyboard shortcuts](#) and every [feature](#) with examples.

Make it yours

The [Settings window](#), [themes](#) and the complete [configuration file reference](#).

When something is off

[Troubleshooting](#), the [FAQ](#) and what Sevak does with your data in [Privacy](#).

Everything at a glance

You type	What happens	Read more
<code>fire</code>	Finds Firefox; <code>Enter ↵</code> launches it	Applications
<code>12*7, 5 km in mi</code>	Shows the answer; <code>Enter ↵</code> copies it	Calculator and conversions
<code>g rust traits</code>	Searches Google in your browser	Web search
<code>f report</code>	Finds files in your chosen folders	Files and folders
<code>~/Downloads/ , C:\Users\</code>	Browses folders as you type	Files and folders
<code>b recipes</code>	Opens a browser bookmark	Browser bookmarks
<code>cb</code>	Shows recent clipboard entries (once clipboard history is turned on)	Clipboard history
<code>s sig</code>	Pastes a saved snippet	Snippets
<code>lock, sleep</code>	Runs a system command	System commands
<code>> git status</code>	Runs a shell command in a terminal	Shell commands

The exact keywords above are the defaults; most can be changed. The [features overview](#) lists every keyword and how to change it.

Prefer a printed manual?

The whole user guide is also available as a single [PDF](#), generated from these pages.

Getting started

Installing Sevak

Download Sevak for your operating system from the [Releases](#) page.



Unsigned builds

Release builds are currently unsigned. Your OS may show a security warning on first run. This is normal; see the platform-specific instructions below.

Package managers (coming soon)

Packages for winget (`NinadKulkarni.Sevak`), Scoop, Homebrew and the AUR (`sevak-bin`) are prepared but not published yet; progress is tracked in [#12](#). Until then, use the installers below.

Once they are available, Scoop and AUR installs will leave updates to the package manager (Sevak's own update check is off there, and "Check for updates" in the tray names the command to run), while winget and Homebrew installs keep Sevak's own updater, which those package managers recognise.

Windows 10 / 11

1. Download the installer from [Releases](#):
2. `Sevak_<version>_x64-setup.exe` (recommended): Per-user install, no admin rights needed.
3. `Sevak_<version>_x64_en-US.msi` : Per-machine install for enterprise deployments. Needs administrator rights; MSI updates are automatic.
4. Run the installer and follow the prompts.
5. After installation, Sevak appears in your Start Menu and runs in the background at startup.



SmartScreen warning

Windows SmartScreen may show "Windows protected your PC" because the installer is unsigned. Click "More info", then "Run anyway" to proceed.



WebView2 runtime

Sevak uses Microsoft WebView2. On Windows 11 and recent Windows 10 versions it is already installed. If it is missing, the installer automatically downloads it from Microsoft (internet required).

Uninstall: Open Settings → Apps → Installed apps, find Sevak, and click Uninstall. Your config and data in `%APPDATA%\sevak\` are preserved; see [Files and data](#) to delete them.

macOS 11+

1. Download `Sevak_<version>_universal.dmg` from [Releases](#).
2. The .dmg runs natively on Apple silicon and Intel Macs.
3. Open the .dmg and drag Sevak into Applications.
4. Open Applications and click Sevak to launch it.



Gatekeeper warning

macOS blocks the first launch with "Apple could not verify Sevak" because the app is not notarized.

Click **Open Anyway** in System Settings → Privacy & Security (scroll down to find it).

Alternatively, run in Terminal:

```
xattr -dr com.apple.quarantine /Applications/Sevak.app
```



Usage

- **Menu bar:** Sevak lives in your menu bar (top-right), not the Dock.
- **Hotkey:** The default is `⌘ Alt + Space` (Option+Space). Change it in Settings or the config file.
- **File search permissions:** Sevak asks for permission to index Desktop, Documents and Downloads. Decline if you don't use file search; remove those folders in Settings.
- **Pasting:** Clipboard history and snippets require Accessibility permission to paste with Cmd+V. Without it, they copy to the clipboard instead.
- Go to System Settings → Privacy & Security → Accessibility and add Sevak.
- If pasting stops after updating, remove Sevak and re-add it (ad-hoc signed builds are treated as new apps).

Uninstall: Delete `/Applications/Sevak.app`. Config and data remain at `~/Library/Application Support/sevak/`; see [Files and data](#) to delete them.

Ubuntu 22.04+ / Debian

Download `Sevak_<version>_amd64.deb` from [Releases](#), then install:

```
sudo apt install ./Sevak_<version>_amd64.deb
```



Start Sevak from your application menu, or run:

```
sevak
```

Uninstall:

```
sudo apt remove sevak
```

Fedora 39+

Download `Sevak-<version>-1.x86_64.rpm` from [Releases](#), then install:

```
sudo dnf install ./Sevak-<version>-1.x86_64.rpm
```

Uninstall:

```
sudo dnf remove Sevak
```

AppImage (other distributions)

Download `Sevak_<version>_amd64.AppImage` from [Releases](#), then run:

```
chmod +x Sevak_<version>_amd64.AppImage  
./Sevak_<version>_amd64.AppImage
```

Requirements

AppImages need FUSE 2 (install `libfuse2` on Ubuntu/Debian, `fuse-libs` on Fedora).

If you cannot install FUSE, extract and run instead:

```
./Sevak_<version>_amd64.AppImage --appimage-extract-and-run
```

The AppImage is built on Ubuntu 22.04 and requires glibc 2.35 or newer.

Permanent installation

Install the AppImage somewhere permanent (not `/tmp` or a temporary folder) before using `sevak --setup-hotkey`, since desktop shortcuts store the full path to the executable.

Setting up the hotkey on Linux

X11 sessions

Sevak registers hotkeys directly. Your configured hotkey (default  + ) works out of the box.

Wayland sessions

Applications cannot grab global keys on Wayland. Sevak provides `sevak --setup-hotkey` to ask the desktop to handle keys:

On GNOME:

```
sevak --setup-hotkey          # use hotkey from config.toml
sevak --setup-hotkey "Ctrl+Space" # or choose a different key
```

This creates GNOME custom keyboard shortcuts for:

- Main hotkey (`[general] hotkey`) → `sevak --toggle`
- Universal Actions hotkey (`[general] actions_hotkey`) → `sevak --actions`
- Each `[[hotkey]]` entry in `config.toml` → `sevak --query '<text>'` or `sevak --run <id>`

After editing `config.toml`, run `--setup-hotkey` again to update shortcuts. Delete old shortcuts in GNOME Settings → Keyboard Shortcuts if you remove config entries.

On other desktops (KDE, Sway, etc.):

Bind a key to `sevak --toggle` in your desktop's keyboard settings. The `--setup-hotkey` command only works on GNOME.

GNOME Alt+Space conflict

GNOME binds  +  to the window menu by default, which blocks Sevak's hotkey.

Choose a different key (e.g.  + ,  + ,  +  + ) or free  + 

```
gsettings set org.gnome.desktop.wm.keybindings activate-window-menu "[]"
```

Note:  +  switches input sources if you have multiple keyboard layouts configured.

Wayland rendering

By default, Sevak uses XWayland (X11 compatibility layer) on Wayland to position itself and take focus reliably. You can use the native Wayland backend instead by setting `[linux] wayland_use_xwayland = false` in your config, though the window may appear off-centre or not accept input.

Tray icon on GNOME (Ubuntu and Fedora)

Sevak displays a tray icon through the AppIndicator protocol.

Ubuntu: The AppIndicator extension is enabled by default. You should see the tray icon.

Fedora: Install the AppIndicator extension:

```
sudo dnf install gnome-shell-extension-appindicator
```

Then enable it in GNOME Extensions or at extensions.gnome.org, or restart GNOME Shell:

```
killall -HUP gnome-shell
```

Without a tray: Sevak still runs. Use the hotkey to show it, and run `sevak --quit` to exit. Open Settings with `sevak --settings`.

Red Hat Enterprise Linux 9 and clones (Rocky, AlmaLinux)



Not supported on RHEL 9 / Rocky 9 / AlmaLinux 9

Prebuilt packages do not work on EL9. Sevak uses Tauri v2, which requires WebKitGTK 4.1 (`webkit2gtk4.1`). EL9 distributions only ship WebKitGTK 4.0 (`webkit2gtk3`), which is not compatible and is not available in any EL9 repository (BaseOS, AppStream, CRB, or EPEL 9).

The AppImage also fails to run on EL9 (compiled against glibc 2.35, EL9 has 2.34).

Workarounds on EL9:

- Run Sevak in a **Fedora or Ubuntu container** (e.g. distrobox) with display access.
- **Build from source** on a Fedora or newer distribution.
- Use **Fedora or Ubuntu** directly.
- Wait for **EL10 with EPEL**: AlmaLinux 10 reportedly ships `webkit2gtk4.1` in EPEL 10, so the `.rpm` is expected to work on EL10 clones.

Your first five minutes with Sevak

A quick walkthrough to get you searching and launching.

1. Install and open

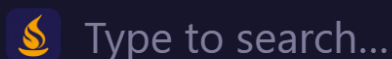
Download from [GitHub Releases](#) and follow the [platform-specific instructions](#).

On **Linux Wayland**, configure your desktop hotkey first: [Setting up the hotkey on Linux](#).

Open Sevak once. You'll see a tray or menu-bar icon that keeps it always accessible.

2. Bring up the search bar

Press `⌘ Alt + Space` (`⌥ Option + Space` on macOS) to open the launcher.



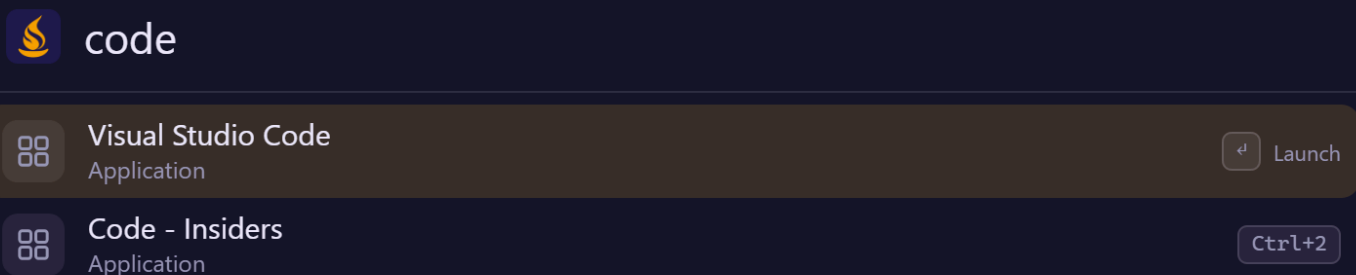
Type immediately—the cursor is already in the search box. Press `⌘ Esc` to hide it.

If `⌘ Alt + Space` is already used by another app, open **Settings** from the tray and change it.

3. Launch an app

Type the name of an installed app: `code`, `firefox`, `vs code`.

Use `↑ Up` / `↓ Down` to pick the right result, then press `Enter ↵` to launch.



Sevak learns what you use most and ranks results by frequency and recency. Recent searches reappear first.

4. Do a quick calculation

Type `12*7` and press `Enter ↵` to copy the result (`84`).

Paste it anywhere. Try `sqrt(16)`, `2^10`, `(125+75)/4`, or `10 km in mi`.

 **12*7**

 **84**
12 * 7 · Enter to copy

 Copy

Sevak handles arithmetic, unit conversion, and more. See [Calculator](#).

5. Find a file or search the web

Query	Result
<code>f project</code>	Finds files/folders matching "project" in your indexed folders (default: Desktop, Documents, Downloads)
<code>g rust traits</code>	Opens Google with that search in your browser
<code>yt svelte</code>	YouTube search
<code>gh tauri</code>	GitHub search

Important: put a space after the keyword (`f` , `g` , `yt` , `gh`).

 **g rust traits**

 Search Google for "rust traits"
<https://www.google.com/search?q=rust%20traits>

 Search

Make it yours

Five quick customizations in **Settings**:

1. **General:** Change the hotkey and set whether Sevak starts at login
2. **Appearance:** Choose Light, Dark, or System theme; adjust font size and window width
3. **Search:** Set the max number of results and fallback web engine
4. **Files:** Add more folders to search

5. **Plugins:** Turn features on or off (Calculator, Bookmarks, Clipboard history, etc.)

Click **Save** when done.

More options: [Configuration guide](#).

Watch it in action

[12-second workflow demo with captions](#) (real interface, sample results).

What's next?

- **Search like a pro:** [Searching and launching](#) — everything you can search and how results are ranked
- **All keyboard shortcuts:** [Keyboard reference](#)
- **Universal Actions:** [Selection-based actions](#) — search, transform, calculate or run actions on text, links or files from any app
- **Deep customization:** [Configuration guide](#) — hotkeys, themes, custom search engines and more
- **Stuck?** [Troubleshooting](#) or [FAQ](#)

Enjoy! Sevak is here to save you time.

User guide

Searching and launching

Learn how Sevak works: opening it, typing queries, selecting results and running actions.

Opening and closing

Show the launcher

Press your global shortcut (default: `⌘ Alt + Space`, `⌥ Option + Space` on macOS). The search bar appears centered on your screen.

On **Linux Wayland**, configure the shortcut first: [Setting up the hotkey](#).

Hide the launcher

- Press `⏏ Esc` while the launcher is open
- Click outside the search bar (if "Hide when focus is lost" is on in Settings)
- Run a result and "Hide when focus is lost" is enabled

Hiding does not quit Sevak. It stays in the background until you press the hotkey again.

Tray menu and system integration

Click the tray icon (Windows, Linux) or menu-bar icon (macOS) to access:

Menu item	Action
Show	Open the launcher
Settings	Open the Settings window
Reload index	Re-read config, rebuild app/file/bookmark indexes, load new script plugins
Check for updates	Look for a new version on GitHub
Quit	Exit Sevak

Searching

Typing a query

The cursor is in the search box as soon as Sevak opens. Start typing immediately—no need to click.

Sevak searches as you type. Results update in real time.

How results are ranked

Sevak combines three factors:

1. **Query match:** how well your typing matches a result (fuzzy matching allows typos and abbreviations)
2. **Frequency:** how often you have run that result recently
3. **Recency:** how long ago you last ran it

Results using frequently and recently rank higher. This means your most-used apps and files appear first, even with a short query.

Examples: - Type `cod` → sees `code` (VS Code), `codeserver`, and `mycode.txt`; if you use VS Code most, it ranks first - Type `pro` → apps like `Prometheus` mix with files like `project.md`, `proposal.docx`; your recent choices decide the order

New installations: on first run, all results are equally ranked. As you use Sevak, personalization kicks in.

Query history

On an **empty search bar**, press `↑ Up` and `↓ Down` to step through your last 50 searches, newest first.

Typing leaves history. Toggle this on or off in **Settings** → **Search** → **Remember searches** or with `[search] query_history`.

Viewing results

The launcher shows a list of matching results. Each row displays:

- **Icon:** the app, file type, or plugin icon
- **Title:** the name of the app, file, or result
- **Subtitle:** additional info (folder path, app description, calculation result, etc.)
- **Action hint:** what pressing `Enter ↵` will do (Launch, Open, Copy, etc.)

Use `↑ Up` / `↓ Down` or `^ Ctrl` + `P` / `^ Ctrl` + `N` to select a result.

Press **++pageup++** or **++pagedown++** to jump by a page (7 results).

Running a result

Press  to run the selected result:

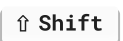
- **Apps:** launch the application
- **Files:** open with the default application (or folder in file manager)
- **Web search:** open the URL in your browser
- **Calculations:** copy the result to your clipboard
- **Clipboard history:** paste the text into the app that had focus before Sevak opened
- **Snippets:** paste the template (with placeholders filled) into the previous app
- **Custom results:** defined by each plugin

Running from keyboard numbers

Press  +  through  +  to run the corresponding result in the list without arrow keys. The numbers match the rows 1–9.

Result actions

Most results can be used in more than one way. The row at the bottom of the list shows what the modifier keys do:

Shortcut	Effect
	Run the primary action (Launch, Open, etc.)
 + 	Alternative action bound to Ctrl, if available
 + 	Alternative action bound to Shift, if available
 + 	Alternative action bound to Alt, if available

Action panel

Press  (with the cursor at the end of the text) or  +  to open the action panel, which lists **every available action** for the selected result.

In the action panel:

-  /  or  +  /  +  : select an action
-  : run the selected action
-  +  through  +  (Universal Actions only): run the numbered action

- `Esc` or `← Left` or `^ Ctrl + K` : close the panel

Copying and Large Type

Copy a result: press `^ Ctrl + C` (when nothing is selected in the text input) to copy the result's value, path, or URL to your clipboard.

Large Type: press `^ Ctrl + L` to show the selected result's text huge across the screen. Press any key to dismiss it. Useful for reading: - Long file paths - URLs - Calculation results - Large text from the selection (Universal Actions)

Tab completion

Press `Tab →` to complete the input to the selected result's name or value. The text in the search box is replaced.

Examples: - Searching apps: **Tab** fills in the selected app's full name - File browsing: **Tab** completes to the selected file or folder name (folders get a trailing slash so you can keep drilling) - Web search: **Tab** completes to `keyword` (the keyword with a space, ready for your search terms)

When browsing a folder path, `↑ Shift + Tab →` goes back up one level.

Plugins and features

Sevak includes several built-in result sources. Each can be turned on or off in **Settings** → **Plugins**:

Applications

Type part of an installed application's name. Fuzzy matching lets a short query find a longer name.

Sources differ by platform: - **Windows**: Start Menu shortcuts and installed packages - **macOS**: application bundles - **Linux**: desktop entries

After installing a new app, choose **Reload index** from the tray menu if it doesn't appear immediately.

More: [Applications](#) plugin.

Calculator

Enter an expression directly; no keyword is required. Results update as you type.

`Enter ↵` copies the number to your clipboard.

Examples: - Arithmetic: `12*7`, `(125+75)/4` - Powers: `2^10` - Functions: `sqrt(16)`, `sin(1.5)`, `ln(100)` - Factorial: `5!` - Remainder: `17%5`

Unit conversion

Type `<amount> <unit> (in|to|as|=) <unit>`. The amount can be any expression: `(2+3) km in m`.

Supported: length, mass, temperature, volume, area, speed, data, time, pressure, energy, angle.

Examples: - `10 km in mi`, `5'11" to cm`, `3 ft 4 in to cm` - `100°F in C`, `0 c to k` - `1 kg in lb`, `8 oz to g` - `5 GB in MiB`, `100 Mbit to MB` - `90 min to h`

For data: `MB` is megabytes (1,000,000 bytes); `MiB` is mebibytes (1,048,576 bytes). `KB` / `MB` / `GB` are powers of 1000; `KiB` / `MiB` / `GiB` are powers of 1024.

Currency conversion

Optional and off by default (requires network). Enable in **Settings** → **Plugins** → **Currency conversion**.

When on, `100 usd in eur` works. Use ISO codes, signs or words: `100 usd in eur`, `50 € to $`, `$100 in eur`.

The subtitle shows the exchange rate and the European Central Bank's publication date.

Rates are downloaded at most once per day and cached locally.

More: [Calculator](#) plugin.

Files and folders

Type `f` (keyword + space) followed by a filename or folder name to search indexed directories.

Sevak searches **names only**, not document contents. Default roots: Desktop, Documents, Downloads.

By default, dot-files and cache folders (`node_modules`, `.git`, etc.) are excluded.

Browsing a path: type a full path (e.g., `~/Documents/`, `/etc/`, `C:\Users\`) to list a folder directly. **Tab** completes entries; `↑ Shift + Tab →` goes up; `Enter ↵` opens.

Configure in **Settings** → **Files** or with `[files]` in `config.toml`.

More: [Files](#) plugin.

Bookmarks

Type `b` and part of a bookmark's title or URL to search bookmarks from Chrome, Edge, Firefox, Safari and other browsers.

Sevak reads your bookmarks across all profiles, read-only.

`Enter ↵` opens the URL in your default browser.

More: [Bookmarks](#) plugin.

Clipboard history

Optional and off by default. Enable in **Settings** → **Plugins** or with `[clipboard] enabled = true`.

Type `cb` to search recent copied text. Results you paste into your previous app with `Enter ↵`.

Sevak never records content password managers mark as secret, copies made in apps listed in `ignore_apps`, very long text, or images.

More: [Clipboard history](#) plugin.

Snippets

Type `s` and a snippet name to search your saved text templates. `Enter ↵` pastes the template (with placeholders filled in) into your previous app.

Placeholders: - `{date}`, `{time}`, `{datetime}`: today, current time, both - `{date:FORMAT}`: custom [strftime](#) format, e.g. `{date:%d %B %Y}` - `{clipboard}`: the current clipboard text - `{uuid}`: a new random UUID - `{{ , }}`: literal `{` and `}`

Configure in `[paste]` and with `[[snippet]]` entries in `config.toml`.

More: [Snippets](#) plugin.

System commands

Type a command's name: `lock`, `sleep`, `restart`, `shutdown`, `logout`, `empty trash`, or settings pages (`bluetooth`, `display`, `wifi`, `sound`, etc.).

Dangerous actions (restart, shut down, logout, empty trash) ask for confirmation first. You can disable the ask or hide commands in **Settings** → **Plugins** or with `[system]` in `config.toml`.

More: [System commands](#) plugin.

Shell commands

Type `>` (or `>`) followed by a shell command. `Enter ↵` opens your terminal and runs it.

`>` (space only) shows your recent commands and an "Open terminal" row.

Nothing runs until you press Enter—the launcher only prepares the command.

Terminal auto-detection by platform; customize in **Settings** or with `[shell]` in `config.toml`.

More: [Shell](#) plugin.

Web search

Type a **keyword and your terms**: `g rust traits`, `yt svelte tutorial`, `gh tauri`.

Built-in keywords: - `g` : Google - `yt` : YouTube - `gh` : GitHub

`Enter`  opens the constructed URL in your browser.

When a plain query has no results, a fallback web search is offered (default: Google). Configure or add custom engines in **Settings** → **Web search** or with `[[web_search]]` in `config.toml`.

More: [Web search](#) plugin and [adding custom engines](#).

Script plugins

Add your own keywords without building Sevak. Put a folder with `plugin.toml` and a script in the `plugins` folder next to `config.toml`, then **Reload index**.

Scripts run with your permissions (not sandboxed), so install only plugins you trust.

Three examples are in `examples/plugins/`. The full guide is [Plugins](#).

Universal Actions

Select something in any app—text, a link, files—press `^ Ctrl` + `⌘ Alt` + `Space` (or your configured hotkey) and Sevak shows what you can do with it.

Reading the selection

Sevak reads your selection by:

1. Saving your clipboard
2. Pressing `^ Ctrl` + `C` (`⌘ Cmd` + `C` on macOS) in the app you were using
3. Waiting for the copy (up to ~0.3 seconds)
4. Reading the text or files
5. Putting your clipboard back

Nothing is stored. The selection is never logged or recorded. Your OS clipboard history (Windows Win+V, etc.) or a clipboard manager *might* see it because the app itself makes the copy.

On **terminal windows** (Windows Terminal, xterm, etc.), Sevak skips the copy to avoid interrupting a running program; use the mouse and the clipboard fallback instead.

On **Wayland**, app-to-app clipboard reading is not possible. Enable **Use the clipboard if the selection can't be read** in Settings to act on the clipboard instead.

What you can do

Selection	Actions
Text	Search with each web engine, show as Large Type, copy, paste as plain text, calculate (if it's a math expression or unit conversion), transform: Uppercase, Lowercase, Title Case, Trim, URL-encode, URL-decode, Base64 encode, Base64 decode, Pretty-print JSON, Minify JSON
URLs (<code>http://</code> , <code>https://</code> , <code>mailto:</code> , <code>www.</code>)	Open, copy, Large Type
Files and folders	Open, show in file manager, copy path(s), open in terminal (folders), run as admin (Windows programs), send to Sevak (fills search box with the path)
Path as text	File actions above, then text actions

Running actions

Actions are shown in the panel. Use:

- `↑ Up` / `↓ Down` : select an action
- `Enter` : run the selected action
- `^ Ctrl` + `1` through `^ Ctrl` + `9` : run the numbered action directly
- `^ Ctrl` + `Enter` : run the action but copy instead of pasting/replacing
- `↑ Shift` + `Enter` (web search): copy the URL instead of opening it
- `⌫ Esc` : close the panel and return to the launcher

Where pasting works (Windows, macOS outside Wayland, Linux X11): - Transformations and calculator results **replace the selected text** in the app - `^ Ctrl` + `Enter` copies instead of pasting

Where pasting doesn't work (macOS without permission, Wayland): - Results are **copied** only

More: [Universal Actions](#).

Keyboard shortcuts

Complete reference: [Keyboard shortcuts](#).

Quick summary:

Context	Shortcut	Action
Launcher	 Alt + 	Show or hide (or your configured hotkey)
Navigation	 Up /  Down or  Ctrl +  P /  Ctrl +  N	Previous / next result
Running	 Enter 	Run selected result
Actions panel	 → Right or  Ctrl +  K	Open actions panel
Panel	 Up /  Down	Select action
Completion	 Tab 	Complete to selected result
History	 Up /  Down (empty bar)	Recall searches
Copying	 Ctrl +  C	Copy result value/path/URL
Large Type	 Ctrl +  L	Show as Large Type
Close	 Esc	Hide launcher or close panel

Command line

These assume `sevak` is on your path; otherwise use the full path to the executable.

```
sevak                                # Open the launcher
sevak --toggle                       # Toggle the launcher (open or hide)
sevak --query "> "                   # Open the launcher with text pre-filled
sevak --run system:lock              # Run a result by ID without showing the launcher
sevak --actions                     # Universal Actions for the current selection
sevak --background                  # Start without showing the launcher
sevak --settings                    # Open Settings
sevak --quit                        # Quit Sevak
sevak --setup-hotkey                # Configure GNOME shortcuts on Linux
sevak --setup-hotkey Ctrl+Space     # Set up a specific key on Linux
sevak --config ~/Dropbox/sevak      # Use another config folder
sevak --help
sevak --version
```

Only one instance runs at a time; subsequent invocations forward their action.

See also

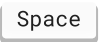
- [Keyboard reference](#) — every key Sevak handles
- [Universal Actions in depth](#)
- [All plugins and features](#)
- [Configuration guide](#)

Keyboard shortcuts

Complete reference to every key Sevak handles.

Global hotkeys

These work anywhere on your system:

Shortcut	Action	Configure
 + 	Show or hide the launcher	<code>[general] hotkey</code>
 +  + 	Universal Actions for your current selection	<code>[general] actions_hotkey</code> (or empty to disable)
Custom <code>[[hotkey]]</code> entries	Type a query or run a result without showing the launcher	<code>[[hotkey]]</code> in config.toml or Settings → Hotkeys

On **macOS**,  is , and  is .

On **Linux Wayland**, apps cannot register global keys; run `sevak --setup-hotkey` to bind the launcher shortcut in GNOME. For the effect of a custom `[[hotkey]]` there, bind a desktop shortcut to `sevak --query "<text>"` or `sevak --run <result-id>` yourself (see [Command line](#)).

In the launcher window

Navigation and selection

Key	Action
 or  + 	Previous result
 or  + 	Next result
<code>++pageup++</code>	Previous page (7 results)
<code>++pagedown++</code>	Next page (7 results)

Key	Action
<code>Tab →</code>	Complete the input to the selected result's name/value
<code>↑ Shift</code> + <code>Tab →</code>	Go up one folder while browsing a path

Running results and actions

Key	Action
<code>Enter ↵</code>	Run the selected result
<code>^ Ctrl</code> + <code>Enter ↵</code> , <code>↑ Shift</code> + <code>Enter ↵</code> , <code>⌘ Alt</code> + <code>Enter ↵</code>	Run the selected result's alternative action bound to that modifier, if it has one. The row shows which ones it offers (for example <code>Ctrl+␣</code>), and each feature page lists them
<code>^ Ctrl</code> + <code>1</code> to <code>^ Ctrl</code> + <code>9</code>	Run the 1st to 9th result in the list directly

Actions panel

Press `→ Right` (with cursor at the end of text) or `^ Ctrl` + `K` to open; shows every action of the selected result or your selection.

Key	Action
<code>↑ Up</code> or <code>^ Ctrl</code> + <code>P</code>	Previous action
<code>↓ Down</code> or <code>^ Ctrl</code> + <code>N</code>	Next action
<code>Enter ↵</code>	Run the selected action
<code>^ Ctrl</code> + <code>1</code> to <code>^ Ctrl</code> + <code>9</code>	Run the corresponding action (Universal Actions only)

Key	Action
 or  +  or 	Close the panel (in Universal Actions,  hides Sevak, as there is nothing behind the panel)
Any other key	Closes the panel and acts as usual (typing goes to the search bar)

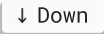
When using Universal Actions, each action's row shows its modifiers ( ,  , ) for alternative behavior.

Display and copying

Key	Action
 + 	Show the selected result as Large Type across the screen (dismiss with any key)
 + 	Copy the selected result's path, URL, or value (when nothing is selected in the text input)

Search history

On an **empty search bar**:

Key	Action
	Step to the next earlier search (most recent first)
	Step to the next later search

Typing leaves history. Toggle history on or off with `[search] query_history`.

Closing

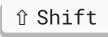
Key	Action
	Hide the launcher (or close the actions panel)

Settings window

General navigation

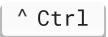
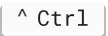
Key	Action
 + 	Save the current settings
 or 	Move between tabs (when a tab button has focus)
 or 	Jump to the first or last tab

Form controls

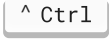
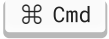
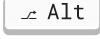
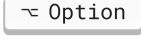
-  /  +  move between controls
-  toggles a switch or checkbox
- Arrow keys change a slider or the selected option in a group

Blocked keys

The following are blocked in production to prevent accidental page reloads:

Key	Context
	Launcher and Settings
 + 	Launcher and Settings
 + 	Launcher and Settings
 + 	Settings (print); in the launcher  +  means "previous result"

Platform differences

Function	Windows/Linux	macOS
Ctrl prefix		
Alt prefix		
Tray access	Click the icon; no key	Click the menu-bar icon; no key

Custom shortcuts

Add global hotkeys in your config file or Settings → Hotkeys:

```
[[hotkey]]
key = "Ctrl+Alt+T"
query = "> "                # open Sevak with this text

[[hotkey]]
key = "Ctrl+Alt+L"
run = "system:lock"        # run this without showing the launcher
```

See [Custom hotkeys](#) for details.

Settings window

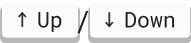
Configure Sevak through the Settings window.

Open Settings

- Click the tray or menu-bar icon and select **Settings**
- Run `sevak --settings` from the command line

Using Settings

Each tab covers a different aspect of Sevak. Changes are **not saved** until you click **Save**. Click **Close** or **Cancel** to discard them.

Control	Result
	Save settings
 on a tab button	Move between tabs
 on a tab button	Jump to first or last tab

Validation errors appear under each field and prevent saving until they are fixed.

General

Shortcut

The global hotkey to show and hide Sevak. Tap it to toggle the launcher from anywhere.

Examples:  ,  ,  . The key combination must not be in use by another app or your desktop.

On **Linux Wayland**, the desktop manages global hotkeys. If the field won't register, click **Set up GNOME shortcut** to create a desktop binding, or run `sevak --setup-hotkey "Your+Combo"` .

Links to: `[general]` hotkey in configuration.

Universal Actions shortcut

Global hotkey to capture and act on what you have selected in another app: text, a link, or files.

Leave **empty** to turn off Universal Actions.

On **Wayland**, use the same **Set up GNOME shortcut** button.

Links to: `[general] actions_hotkey` in configuration.

Use the clipboard if the selection can't be read

Wayland, terminal windows and some apps prevent Sevak from reading the selection. Turn this on to act on the clipboard instead when the direct method fails.

Links to: `[actions] use_clipboard_fallback` in configuration.

Hide when focus is lost

Automatically close the launcher when you click outside it or switch to another window.

Links to: `[general] hide_on_blur` in configuration.

Launch at login

Start Sevak in the background when you sign in.

Links to: `[general] launch_at_login` in configuration.

Check for updates

Look for a new version at startup and once per day. If an update is available, you are asked before it is installed.

Links to: `[general] check_for_updates` in configuration.

Hotkeys

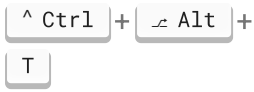
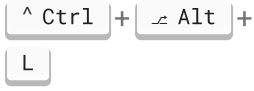
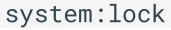
Define custom global hotkeys that open Sevak with text already typed, or run a result without showing the launcher.

Adding a hotkey

Click **Add hotkey** and fill in:

- **Key:** the shortcut (e.g., `^ Ctrl` + `⌘ Alt` + `T`)
- **Query** or **Run:** either type text to pre-fill the search bar, or give a result ID to run immediately

Examples

Hotkey	Action	Result
	Query 	Opens Sevak with  typed (ready for a shell command)
	Run 	Locks the screen without showing the launcher

Finding result IDs

Hover over any result for a moment to see its ID. Examples: - `apps:firefox.desktop` (Linux app) - `apps:Mozilla Firefox` (Windows app) - `system:lock`, `system:settings:bluetooth` - `files:/Users/me/Documents` - `shell:git status`

Calculator, web search and clipboard history results have no stable ID and cannot be bound.

Wayland: custom hotkeys are registered as GNOME shortcuts. Delete them in GNOME's keyboard settings when you remove an entry.

Links to: `[[hotkey]]` in [configuration](#).

Appearance

Theme

- **System:** follow your operating system's light/dark setting
- **Light:** always light
- **Dark:** always dark

Links to: `[appearance] theme` in [configuration](#).

Window width

The width of the search bar in pixels. Range: 400–1600 px.

Links to: `[window] width` in [configuration](#).

Accent color

Highlight color for buttons, toggles and the caret. Format: `#rgb`, `#rrggbb`, or `rgb(r, g, b)`. Leave empty to use the theme's default.

Examples: `#7c3aed`, `#fa0`, `rgb(124, 58, 237)`.

Links to: `[appearance] accent` in configuration.

Font size

Size of the result titles in pixels. Range: 12–22 px. The rest of the search bar scales with it.

Links to: `[appearance] font_size` in configuration.

Font family

Font for the launcher. Leave empty to use your system's default. Examples: `Fira Sans`, `sans-serif`, `Monospace`.

Links to: `[appearance] font_family` in configuration.

Opacity

Background opacity of the search bar as a percentage. Range: 30–100%. Text stays fully opaque.

Links to: `[appearance] opacity` in configuration.

Corner radius

Rounding of the search bar's corners in pixels. Range: 0–32 px (0 = sharp corners).

Links to: `[appearance] radius` in configuration.

Custom stylesheet

A CSS file in your config folder to override theme variables. Example: `theme.css`. Leave empty to use none.

Sevak's CSS variables are listed in [Themes and appearance](#). Your stylesheet is loaded after the built-in theme, so it overrides everything.

Links to: `[appearance] custom_css` in configuration and [Themes guide](#).

Search

Maximum results

How many rows the result list shows. Range: 1–20.

Links to: `[search] max_results` in configuration.

Fallback web search

Offered when a plain query (not a web keyword) produces no results. Choose from your defined web search engines, or **None** to turn it off.

Examples: If set to `g`, a query like `rust traits` that has no app or file results shows a Google search row.

`config.toml` can list several fallbacks; the dropdown shows them. If you pick a different one here, it overwrites the list.

Links to: `[search] fallback_web_search` in configuration.

Remember searches

On an empty search bar, press `↑ Up` and `↓ Down` to step through your last 50 searches, newest first.

Disabling this also deletes the stored search history.

Links to: `[search] query_history` in configuration.

Plugins

Enable or disable built-in result sources:

- **Applications** (Windows Start Menu shortcuts and packages, macOS app bundles, Linux desktop entries)
- **Calculator** (arithmetic, unit conversion, currency)
- **Files** (indexed folders; also see Files tab)
- **Bookmarks** (browser bookmarks; also see Bookmarks section in configuration)
- **Clipboard history** (if enabled; search recent copied text)
- **Snippets** (saved text templates)
- **System commands** (lock, sleep, settings, etc.)
- **Shell** (type `> command` to run in a terminal)
- **Selection** (Universal Actions)
- **Web search engines** (see Web search tab)

Currency conversion

Optional: download the European Central Bank's daily exchange rates to convert currencies.

When enabled, `100 usd in eur` works in the calculator. Rates are cached locally for 24 hours. Disable to keep Sevak offline.

Links to: `[calculator] currency` in configuration and [Privacy](#).

Web search

Define keywords and URL templates for web search.

Adding an engine

1. Click **Add engine**
2. Fill in:
3. **Keyword:** what you type (e.g., `g`, `yt`, `gh`)
4. **Name:** displayed name (e.g., `Google`, `YouTube`)
5. **URL:** search URL with `{query}` placeholder (e.g., `https://www.google.com/search?q={query}`)

Type the keyword and your search terms to use it: `g rust traits` opens Google with those terms.

Example engines

Keyword	Name	URL
<code>g</code>	Google	<code>https://www.google.com/search?q={query}</code>
<code>yt</code>	YouTube	<code>https://www.youtube.com/results?search_query={query}</code>
<code>gh</code>	GitHub	<code>https://github.com/search?q={query}</code>

Default engines

Sevak starts with Google, YouTube and GitHub. You can remove them and add your own.

Links to: `[[web_search]]` in [configuration](#).

Files

Folders to search

Directories and their subdirectories that are indexed and searchable.

Click **Add folder...** to browse and select. Click the **×** to remove.

The index is capped at 100,000 entries. Keep roots focused on your active projects.

Links to: `[files] directories` in [configuration](#).

Folder depth

How many levels below each folder are indexed. Range: 0 (just the folder itself) to unlimited.

Default: 4.

Links to: `[files] max_depth` in configuration.

Include hidden files

Index dot-files and dot-folders (e.g., `.git`, `.env`).

Default: off.

Links to: `[files] include_hidden` in configuration.

Keyword

Type "keyword filename" to search only files. Leave empty to disable.

Default: `f`.

Links to: `[files] keyword` in configuration and [configuration guide](#).

Show in global results

Also include matching files in plain searches, ranked lower than apps and bookmarks.

Default: on.

Links to: `[files] global` in configuration.

Linux

Wayland-specific options (appears only on Wayland sessions).

Use XWayland on Wayland

Native Wayland windows cannot position themselves or reliably receive focus, so Sevak draws through XWayland to stay centered and typeable.

Turn this off to use the native Wayland backend (experimental; you may need to position the window manually).

Restart Sevak to apply.

Links to: `[linux] wayland_use_xwayland` in configuration.

Footer buttons

Button	Action
Open config file	Opens <code>config.toml</code> in your default editor
Reveal logs folder	Shows the folder containing Sevak's debug logs
Save	Save and apply all changes (only enabled when valid)
Cancel / Close	Close Settings without saving (or discard changes if you made any)

Validation errors (shown in red) must be fixed before you can save.

Themes and appearance

Sevak follows your system's light or dark mode by default. Everything else is optional and lives in the `[appearance]` table of `config.toml` (the Settings window, Appearance page, edits the same values).

Settings

```
[appearance]
theme = "system"      # "system", "light" or "dark"
accent = ""           # "#rgb", "#rrggbb", "rgb(r, g, b)" or ""; "" uses the theme's accent
font_size = 15        # 12-22 pixels; the size of result titles
font_family = ""      # "Fira Sans, sans-serif"; "" uses the system font
opacity = 100         # 30-100, percent opacity of the search bar's background
radius = 14           # 0-32, corner radius in pixels
custom_css = ""       # a stylesheet in the config folder; "" loads none
```

Changes apply when you save in **Settings**, or after "Reload index" in the tray menu when you edit the file by hand.

Configuration through Settings

All these options are available in **Settings** → **Appearance**. The GUI guides you with hints for each setting.

Validation

Every value is checked when it is applied. A value that is not valid is ignored (that one setting falls back to its default), a warning is written to the log and shown under **Settings** → **Appearance**:

Setting	Valid range	Default
theme	"system", "light" or "dark"	"system"
accent	#rgb, #rrggbb, rgb(r, g, b) or empty	empty (the theme's accent)
font_size	whole number 12-22	15
font_family	names made of letters, digits, spaces and - _ . , separated by commas; up to 200 characters	empty (system font)
opacity	whole number 30-100	100

Setting	Valid range	Default
<code>radius</code>	whole number 0–32	14
<code>custom_css</code>	relative path inside the config folder	empty

Accent color

Setting `accent` derives related CSS variables:

- `--accent-strong` : 18% darker (for hover states)
- `--selected` : accent at 17% opacity (background of selected result)
- `--on-accent` : black or white, whichever reads better on the accent

Opacity and styling

`opacity` fades only the search bar's background; text stays fully opaque. It requires a window that supports transparency, which Sevak uses for rounded corners and shadow anyway.

Sevak does not offer background blur (Acrylic, Mica, vibrancy) because it would blur the transparent margin around the bar, creating a square frame effect.

`font_size` scales the whole search bar (input, subtitles, row height), not only the result titles. It does not affect the Settings window.

`radius` sets corner rounding. `0` = sharp corners; `32` = very rounded.

Custom stylesheet

`custom_css` names a file inside the config folder, for example `theme.css` next to `config.toml` (or `themes/dark-purple.css`). It is a plain CSS stylesheet loaded after Sevak's built-in theme and appearance settings, so it overrides all of them.

Rules for custom stylesheets

- The path must be relative and stay inside the config folder: no absolute paths, no `..`, and a symlink pointing outside is refused
- The file must be UTF-8 and at most 64 KiB; otherwise it is ignored with a warning
- It is read again whenever the config is reloaded (tray menu) or saved (Settings)

- It cannot load anything from the network: Sevak's content security policy blocks remote stylesheets, fonts and images. Use fonts installed on your system and `data:` URLs
- If you keep the config folder in a synced folder, your theme travels with it

Example theme

```
/* theme.css: purple dark theme */
:root {
  --bg: #1b1030;
  --fg: #f1e9ff;
  --muted: #a99bc8;
  --accent: #a78bfa;
  --accent-strong: #8b5cf6;
  --on-accent: #1b1030;
  --selected: rgba(167, 139, 250, 0.2);
}
```

Scoping to light or dark mode

The built-in light and dark values are applied with the same specificity as a plain `:root` rule, and your file comes later, so `:root { ... }` overrides everything.

To change a variable only in one mode:

```
:root[data-theme="dark"] { --bg: #0a0a0a; } /* when theme = "dark" */
:root[data-theme="light"] { --bg: #fafafa; } /* when theme = "light" */
@media (prefers-color-scheme: dark) { /* when theme = "system" and OS is dark */
  :root:not([data-theme]) { --bg: #0a0a0a; }
}
@media (prefers-color-scheme: light) { /* when theme = "system" and OS is light */
  :root:not([data-theme]) { --bg: #fafafa; }
}
```

CSS variables

Set by the light and dark themes (`ui/src/app.css`):

Appearance

Variable	Used for
<code>--bg</code>	Search bar and Settings window background

Variable	Used for
<code>--fg</code>	Main text color
<code>--muted</code>	Secondary text: subtitles, hints, placeholder
<code>--border</code>	Hairlines between sections and form fields
<code>--accent</code>	Caret, buttons, switches, slider thumbs, focus rings
<code>--accent-strong</code>	Hover and focus states of accented elements
<code>--on-accent</code>	Text and icons on an accent background
<code>--selected</code>	Background of the selected result row
<code>--tile</code>	Background behind result icons
<code>--shadow</code>	The search bar's drop shadow (a <code>box-shadow</code> value)

UI elements

Variable	Used for
<code>--kbd-bg</code> , <code>--kbd-border</code>	Key-cap hints (<code>^ Ctrl</code> , <code>K</code> , <code>Enter ↵</code> labels)
<code>--warn</code> , <code>--error</code> , <code>--ok</code>	Notice, error and success message colors
<code>--surface</code>	Panels and sections in the Settings window
<code>--input-bg</code> , <code>--input-border</code>	Text fields and select boxes in Settings
<code>--switch-off</code>	Toggle track when off; slider track

Computed from settings

These are set automatically from appearance settings (override them in your stylesheet if you prefer):

Variable	Source	Default
<code>--font-family</code>	<code>font_family</code> setting	system font
<code>--font-size</code>	<code>font_size</code> setting	15px
<code>--font-scale</code>	<code>font_size</code> / 15	multiplies other text sizes and row height
<code>--radius</code>	<code>radius</code> setting	14px
<code>--card-opacity</code>	<code>opacity</code> / 100	1

Theme examples

Share your theme! Examples could include:

- High-contrast themes for accessibility
- Seasonal themes (holiday colors, etc.)
- Themed to match your wallpaper
- Minimal or dark themes

The class names inside Sevak's UI may change between versions, so override only CSS variables and avoid selectors like `.result` or `.row`.

More customization

- [Custom hotkeys and keyboard shortcuts](#)
- [Complete settings reference](#)
- [Configuration guide](#)

See also

- [Privacy: custom stylesheets cannot load from the network](#)
- [Troubleshooting: appearance warnings and validation errors](#)

Features

Features overview

Sevak includes built-in result sources (plugins) for applications, calculations, files, bookmarks and more. Each plugin answers queries, and the search engine routes and ranks results to show the most relevant ones first.

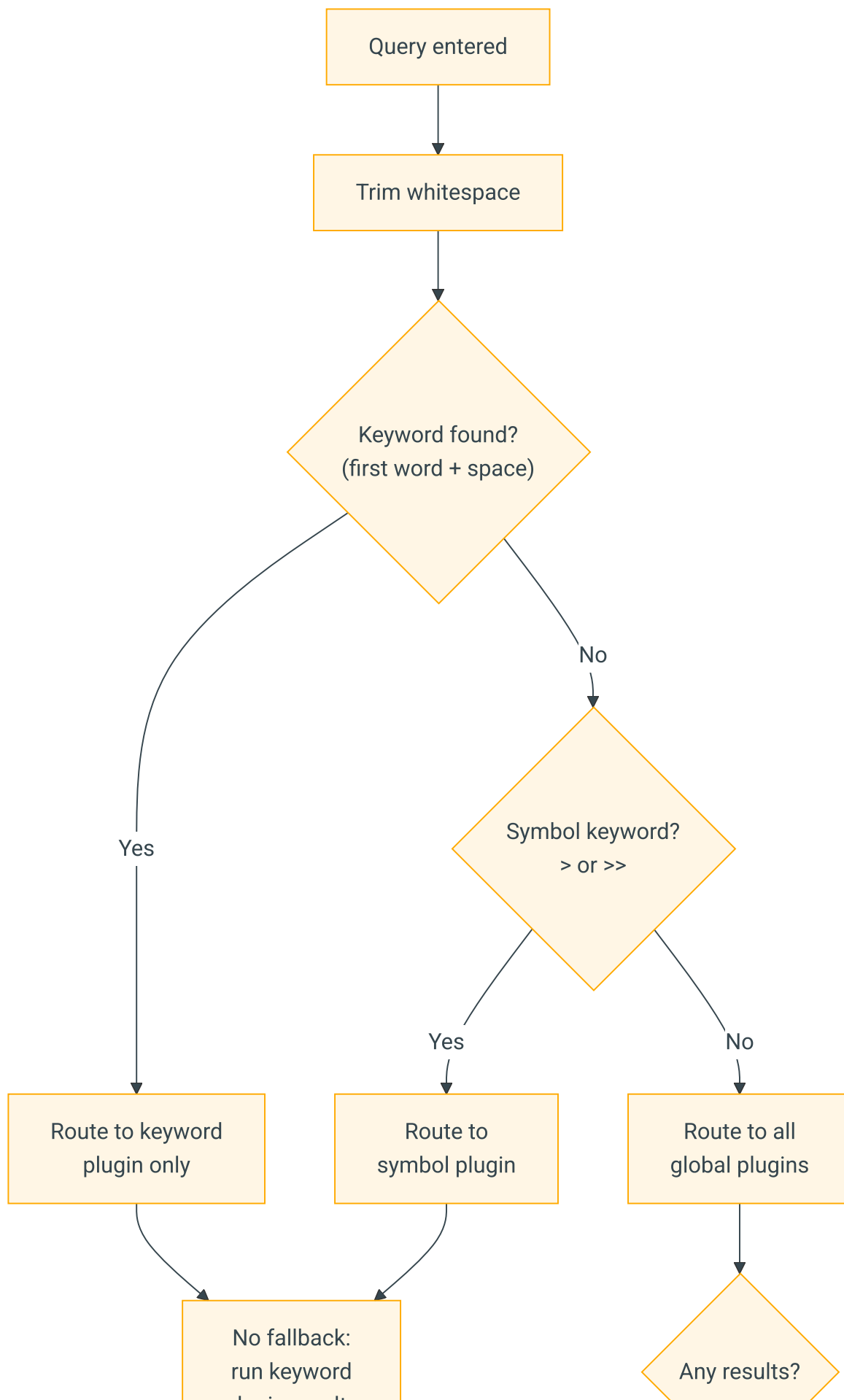
All built-in features

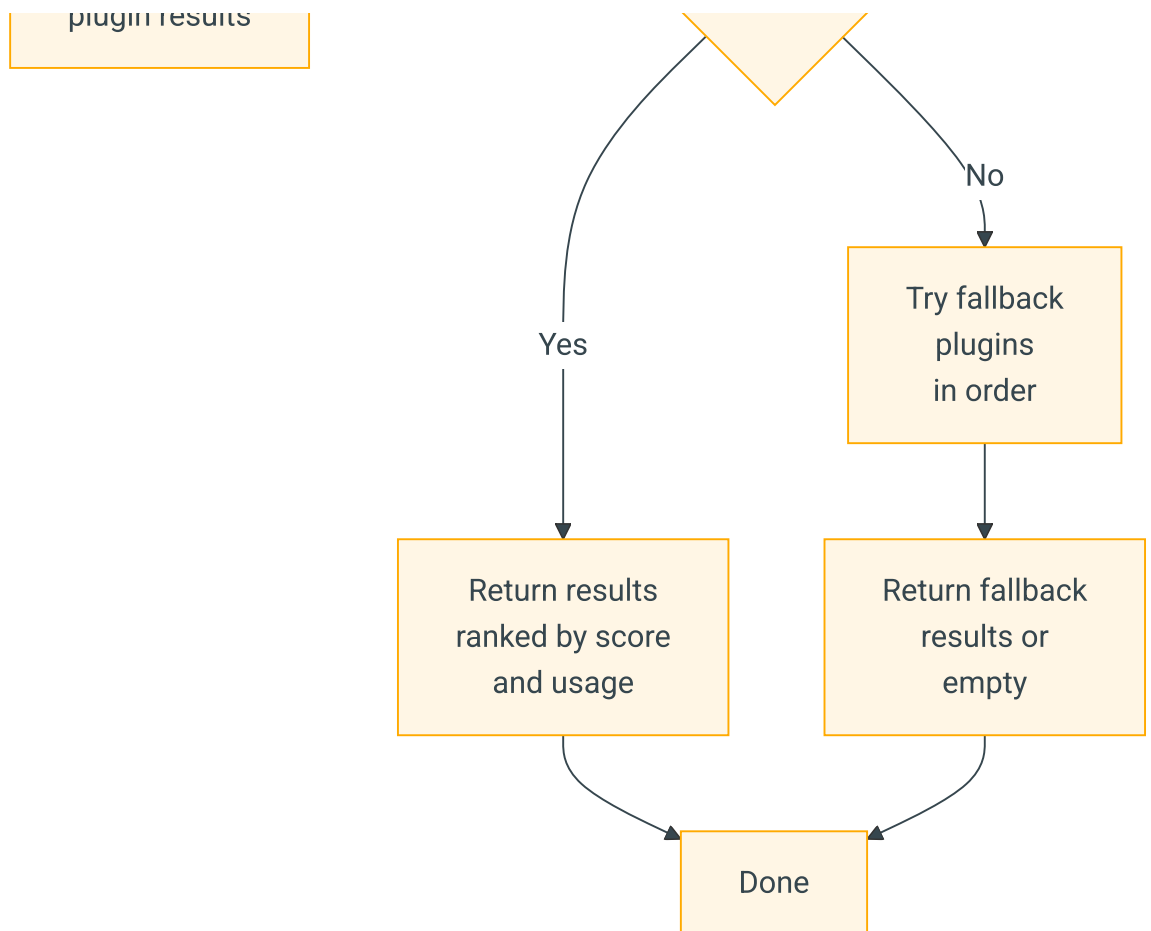
Plugin	Trigger	Keyword	What Enter does	Learn more
Applications	Type any app name	(global)	Launch the application	Applications →
Calculator	Type a math expression	(global)	Copy the result	Calculator →
Web search	<code><keyword></code> <code><search></code>	g, yt, gh	Open search results	Web search →
Files	<code>f <name></code> or path-browse	f	Open the file or folder	Files →
Bookmarks	<code>b <bookmark name></code>	b	Open the URL	Bookmarks →
System commands	Type a command name	(global)	Run the command	—
Terminal commands	<code>> <command></code>	>	Run in a terminal	—
Clipboard history	<code>cb <text></code>	cb	Paste the previous text	—
Snippets	<code>s <snippet></code>	s	Paste the saved text	—

Plugin	Trigger	Keyword	What Enter does	Learn more
Universal Actions	Press hotkey on selection	(hotkey)	Open action menu	—
UUID generator	<code>uuid</code>	uuid	Copy a generated UUID	—

How query routing works

The search engine routes each query to the right plugins using these rules:





Keyword routing

Type `<keyword><space><rest>` to query only plugins with that keyword. For example: - `f documents/report` searches only in Files - `g rust traits` searches only Google - `b github` searches only Bookmarks

The space is optional after symbol keywords: both `> ls` and `>ls` route to Terminal commands.

Global queries

Without a keyword, global plugins answer every query: - **Applications** (global, no keyword) - **Calculator** (global, no keyword) - **System commands** (global, no keyword) - **Files** (global if enabled; `global = true` in config) - **Bookmarks** (global if enabled; `global = true` in config)

Plugins with a keyword but also answering globally (e.g., Files with `f` keyword) have their results multiplied by **0.5** to rank below their dedicated keyword searches.

Ranking

Results are scored by: 1. The plugin's own scoring (e.g., how closely a filename matches the query) 2. Your usage history (frequently picked results rank higher) 3. Query time (results from slower plugins lag behind faster ones)

Exact matches and keyword-triggered results keep the order their plugin gave them. They do not get boosted by usage history because they are already highly relevant.

Fallback

When no global plugin found any results, the search engine tries fallback plugins in order. This catches cases like: - Empty search → show suggestions from configured web engines (default: Google) - No file or app matches → try web search - New plugin answer → still show something useful

Fallback plugins run after all global plugins have answered, and results are shown in the fallback plugins' configured order (not re-ranked).

Keyword hints

Type just the keyword (`g` , `f` , `b`) and the search engine appends that plugin's keyword row, so Tab can complete to `g` , `f` , `b` ready for input.

Turning plugins on and off

In Settings

Open **Settings** → **Plugins**. Toggle any plugin or instance to enable or disable it.

In config

Add plugin ids or instance ids to `[plugins] disabled`:

```
[plugins]
disabled = ["clipboard", "web:yt"] # turn off clipboard history and YouTube
```

Family ids turn off all instances: `disabled = ["web"]` turns off every web engine. Instance ids turn off one instance: `disabled = ["web:g"]` turns off Google only.

To find a plugin's id, hover over a result for a moment—the id appears at the bottom.

Customizing keywords and defaults

Read individual feature pages for options: - [Web search keywords and engines](#) - [Files keyword and folders](#) - [Bookmarks keyword and browsers](#) - [Calculator functions and units](#)

All options are in `[section]` of `config.toml`. After editing, choose **Reload index** from the tray menu or restart Sevak.

Applications

Launch installed applications by typing their name. Sevak finds applications from your system (Start Menu and packaged apps on Windows, application bundles on macOS, desktop entries on Linux) and ranks matches by how closely they fit your query and how often you use them.

How to use it

Type any part of an application's name. Fuzzy matching lets you skip letters:

Input	What you see	What Enter ↵ does
chrome	Google Chrome	Launch Chrome
msg	Microsoft Outlook (messages)	Launch Outlook
calc	Calculator	Open Calculator
vs co	Visual Studio Code	Launch VS Code
fire	Firefox	Launch Firefox

Matching is case-insensitive. The launcher shows the best matches in order; use the arrow keys to choose a different result.

Actions

Key(s)	Action
Enter ↵	Launch the application
^ Ctrl + Enter ↵	Show the application file in your file manager
⇧ Shift + Enter ↵	Copy the application's full path to your clipboard

Key(s)	Action
 Alt + 	Run as administrator (Windows and Linux only; packaged apps cannot be elevated)

Use  ^ Ctrl +  K to open the actions panel and see all available actions.

Tips and troubleshooting

The app doesn't appear: Applications are indexed when Sevak starts. Install the app, then restart Sevak or choose **Reload index** from the tray menu.

Multiple versions of the same app: If you have several versions installed (e.g., both 32-bit and 64-bit), they all appear as separate results.

Packaged apps: On Windows, packaged apps from the Microsoft Store may have long, generated names (like 9P...) in the launcher even if they display nicely in the Start Menu. Pin your favorite packaged apps in the Start Menu to see them with their display names.

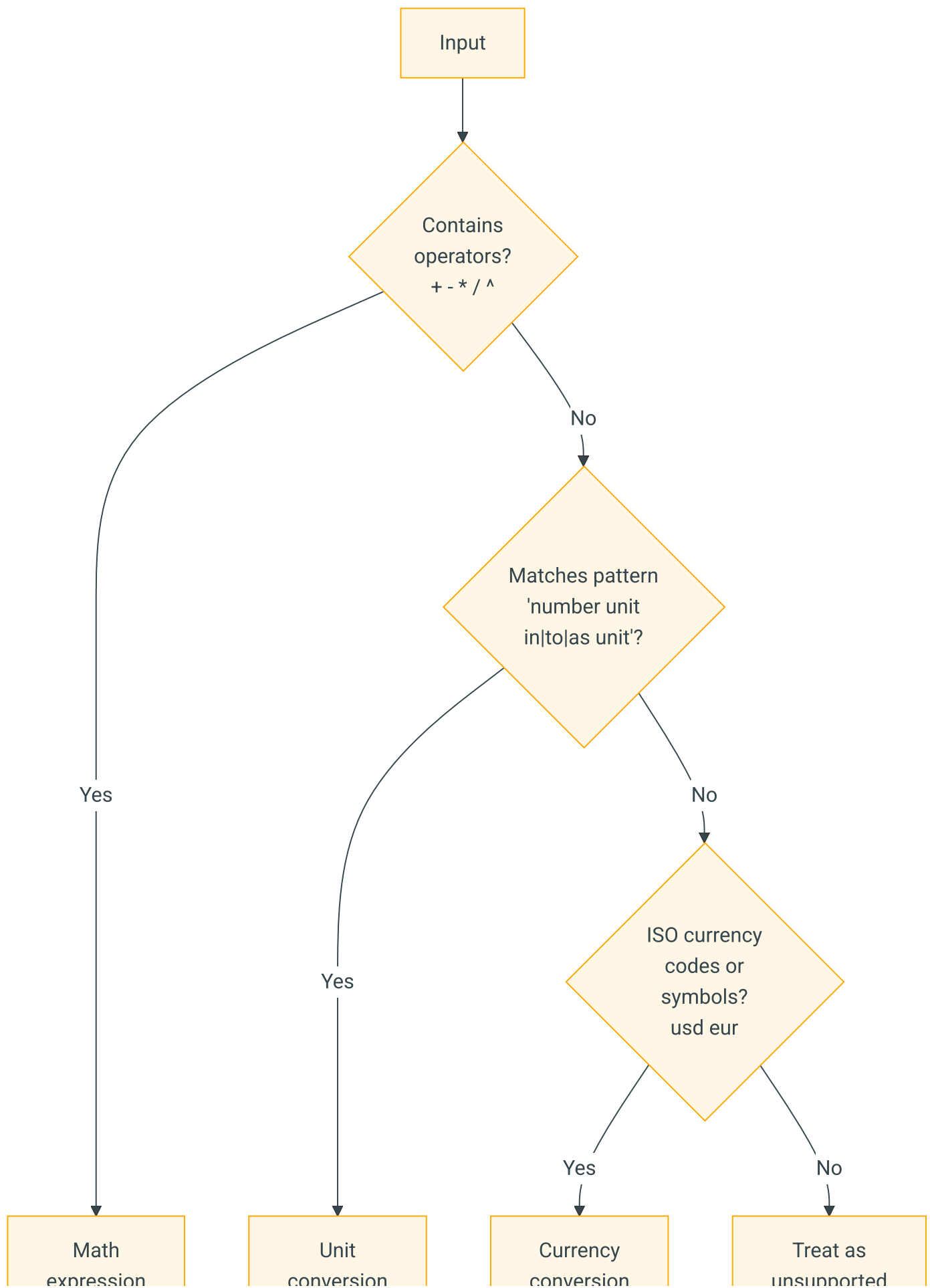
Custom keyboard shortcut: You can bind a key to launch a specific app using [hotkeys in config.toml](#). For example, `run = "apps:firefox.desktop"` on Linux or `run = "apps:Google Chrome"` on Windows.

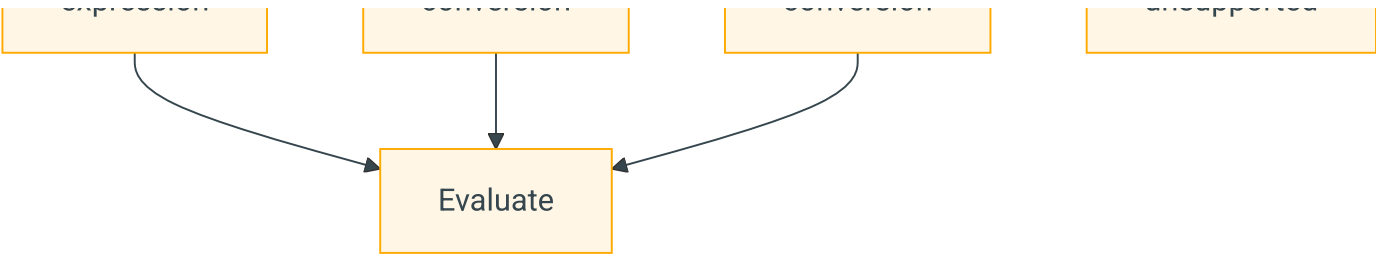
Calculator and conversions

Evaluate math expressions, convert units, and convert currencies without leaving the launcher. No keyword is required; just type.  copies the result.

How to use it

Type a math expression directly into Sevak. The calculator recognizes what you're asking and shows the result:





Math expressions

Type a math expression and `Enter ↵` copies the result. Results use up to 12 significant digits.

Expression	Result	Operation
<code>12*7</code>	<code>84</code>	Multiplication
<code>(125+75)/4</code>	<code>50</code>	Grouped arithmetic
<code>2^10</code>	<code>1024</code>	Powers (<code>^</code> or <code>**</code>)
<code>sqrt(16)</code>	<code>4</code>	Square root
<code>5!</code>	<code>120</code>	Factorial
<code>17%5</code>	<code>2</code>	Remainder (not percentage)
<code>sin(0)</code>	<code>0</code>	Trigonometry

Operators: `+` `-` `*` `/` `%` (remainder) `^` (power, right-associative) `!` (factorial) - `x` and `÷` work as aliases for `*` and `/` - `**` is an alias for `^` - Unary minus: `-2^2 = -4` (minus binds loose; negate then power)

Functions: - Roots: `sqrt(x)` `cbrt(x)` (cube root) - Trig: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh` - Logarithms: `ln` (natural log) `log` (base 10) `log2` `exp` (`e^x`) - Rounding: `floor` `ceil` `round` - Utility: `abs` `sign` `min(a,b)` `max(a,b)` `pow(a,b)` `hypot(a,b)` `atan2(y,x)`

Constants: `pi` (π) `e` `tau` (2π)

Unit conversion

Type `<amount> <unit> (in|to|as|=|→) <target_unit>`.

Input	Result
10 km in mi	6.213711922 mi
72°F to C	22.22222222°C
5'11" to cm	180.34 cm
2 h 30 min in min	150 min
1 acre in m²	4047 m²
60 mph in km/h	96.56064 km/h
5 GB in MiB	4768.371582 MiB

Units work offline. Amounts can be expressions: `(2+3) km in m = 5000 m`.

Supported categories: - **Length:** m, km, cm, mm, mi, yd, ft, in, nmi (nautical mile), ly (light-year), au (astronomical unit) - **Mass:** kg, g, lb, oz (mass ounce), t (metric ton), ton (short ton) - **Temperature:** °C, °F, K (no expression allowed; just `32 f to c`) - **Volume:** L, ml, gal (US gallon), cup, tbsp (tablespoon), tsp (teaspoon), fl oz (fluid ounce), pt (US pint), qt (quart) - Imperial: `imp gal`, `imp pt` for British units - **Area:** m², km², cm², ft², mi², acres, hectares - **Speed:** m/s, km/h, mph (miles per hour), knots, ft/min, any length/time combination (e.g., `mi/h`) - **Data:** B (byte), KB, MB, GB, TB, PB (powers of 1000); KiB, MiB, GiB, TiB, PiB (powers of 1024) - Case matters: `MB` = megabyte, `Mb` = megabit; `kb`, `mb`, `gb` = bytes - **Time:** s, min, h, d, mo (month = 1/12 year), yr (year = 365.25 days) - **Pressure:** Pa, kPa, MPa, bar, atm, psi, torr - **Energy:** J, kJ, MJ, cal, Cal (food Calorie = kcal), kWh - **Angle:** ° `deg` `rad` (radian) `turn` (full rotation)

Compound units: `10 km/h`, `5 sq ft`, `cm²`, `cubic m`, `m/s`, `miles per hour` all work.

Ambiguous cases: - `oz` = mass ounce; `fl oz` = fluid ounce - `m` = meters; `min` = minutes - ° or `deg` alone = angle; °C or `deg C` = temperature - `cal` = small calorie (4.184 J); `Cal` = food Calorie (kcal) - `t` = metric tonne; `ton` = US short ton

Results keep up to 10 significant digits.

Currency conversion

Off by default. Turn it on in **Settings** → **Plugins** → **Currency conversion**, or set `[calculator] currency = true` and reload.

Type `<amount> <currency> in <currency>`. Use ISO codes (`usd`, `eur`) or symbols (`$`, `€`):

Input	Result
100 usd in eur	92.15 EUR (example rate)
50 € to \$	54.50 USD (example rate)
\$100 in eur	92.15 EUR (example rate)

Rates come from the European Central Bank daily, updated at most once a day in the background (never while you type). Until the first download, the result says "Fetching exchange rates...". Amounts can be expressions.

Supported currencies: 30+ major fiat currencies with EUR as the base. The subtitle shows the exchange rate and the ECB's publication date. Cryptocurrencies are not supported.

Options

Setting	Default	What it does	Config section
Currency	false	Enable currency conversion (needs network)	[calculator] currency

Platform notes

Windows

The calculator works identically on all Windows versions. Keyboard layouts that use  as a dead key (European layouts) still parse  as the power operator inside Sevak.

macOS

The calculator works identically on all macOS versions. Type `option+0` for the degree symbol in unit conversions (e.g., 32°F to C).

Linux

The calculator works identically on all Linux desktop environments. Compose key sequences can type special characters outside Sevak; inside the launcher use ASCII equivalents like `deg` for .

Tips and troubleshooting

Percentage of a number: The `%` operator is **remainder**, not percentage. To find 15% of 200, type `200*15/100 = 30`.

Scientific notation: Write `1e3` for 1000, `2e-4` for 0.0002. Leading zeroes and underscores: `1_000_000 = 1000000`.

Rounding noise: `sin(pi)` shows something like `1.2e-16` instead of exactly `0` because of floating-point precision. Real results like `sin(pi/2) = 1` are exact.

Precision: Results are rounded to 12 significant digits to fit the launcher and remove noise from floating-point math. For most everyday use, this is more than enough.

Web search

Search the web with keyword triggers. Sevak comes with Google, YouTube and GitHub pre-configured. `Enter ↵` opens the search results in your browser.

How to use it

Type a keyword followed by a space and your search terms:

Input	Keyword	Search provider	What <code>Enter ↵</code> does
<code>g rust traits</code>	<code>g</code>	Google	Open <code>google.com/search?q=rust+traits</code>
<code>yt keyboard shortcuts</code>	<code>yt</code>	YouTube	Open <code>youtube.com</code> results
<code>gh sevak launcher</code>	<code>gh</code>	GitHub	Open <code>github.com/search</code> results

Press `Tab →` on a bare keyword (`g`, `yt`, `gh`) to complete it with a space, ready for search terms.

The default fallback behavior: when nothing else matches, Sevak searches with the configured fallback engine (by default, Google). You can disable this by setting `[search] fallback_web_search = ""`.

Actions

Key(s)	Action
<code>Enter ↵</code>	Open the search results
<code>↑ Shift</code> + <code>Enter ↵</code>	Copy the search URL to your clipboard

Use `^ Ctrl` + `K` to see all available actions.

Adding custom search engines

Edit `config.toml` to add, remove, or modify web search engines:

```
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"
```

To add a new engine: copy one block and change the `keyword`, `name`, and `url`. The `{query}` placeholder gets replaced with your search terms (percent-encoded).

To remove an engine: delete its entire `[[web_search]]` block.

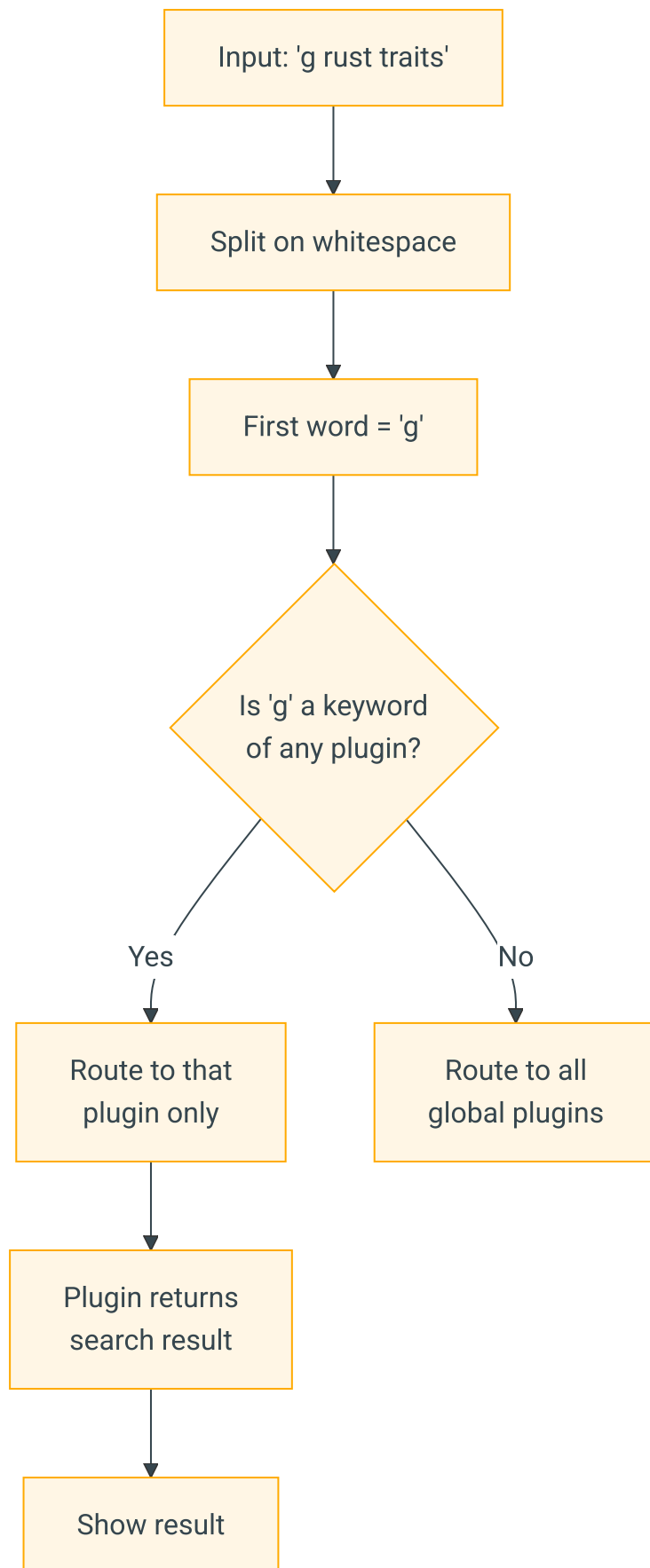
To change the default: define `[[web_search]]` entries in the order they should appear as fallbacks. The first one is used when nothing else matches.

Finding URL templates

Most search engines use `?q=` or `?query=`. Navigate to the search engine's homepage (e.g., [google.com](https://www.google.com)), search for something, and look at the resulting URL. Replace the search term with `{query}`. Examples:

- Google: `https://www.google.com/search?q={query}`
- DuckDuckGo: `https://duckduckgo.com/?q={query}`
- Wikipedia: `https://en.wikipedia.org/w/index.php?search={query}`
- Wiktionary: `https://en.wiktionary.org/w/index.php?search={query}`
- Stack Overflow: `https://stackoverflow.com/search?q={query}`
- npm packages: `https://www.npmjs.com/search?q={query}`
- Wikipedia (full-text): `https://en.wikipedia.org/w/index.php?fulltext=Search&search={query}`

How search routing works



Keyword-triggered web searches are always returned, never down-weighted. When you type `g` with no terms, the engine shows a "Search Google" hint.

Options

Setting	Default	What it does	Config section
Search engines	g, yt, gh	Keywords, names and URL templates for web search	<code>[[web_search]]</code>
Fallback	<code>g</code>	Web engine queried when nothing else matches (empty to disable)	<code>[search]</code> <code>fallback_web_search</code> <code>h</code>

Tips and troubleshooting

Unicode in search terms: Sevak automatically encodes search terms for URLs (spaces become %20, accented characters become UTF-8 bytes, etc.). Type naturally.

Special characters: Symbols like `&`, `#`, `@` are safe to type in search terms and will be encoded correctly.

Multiple fallback options: To offer several fallback engines in order, use a list:

```
[search]
fallback_web_search = ["g", "yt", "gh"]
```



If nothing matches, Sevak tries Google first, then YouTube, then GitHub, in order, returning the first one that gives results.

No search keyword: You can temporarily disable all web search by setting `disabled = ["web"]` in `[plugins]`, but you won't have a fallback for unmatched queries.

Files and folders

Find and open files by name from your configured directories, or browse folders by typing a path. `Enter ↵` opens the file or folder.

How to use it

The Files plugin works two ways: **indexed search** (fast, pre-built) and **path browsing** (live, as you type).

Indexed search

Type `f` followed by at least two characters of a filename:

Input	What you see	What <code>Enter ↵</code> does
<code>f report</code>	All files/folders matching "report"	Open the selected file
<code>f config.toml</code>	Exact match for config.toml	Open config.toml
<code>f src/main</code>	Files in src/ matching "main"	Open the file
<code>report</code> (no keyword)	Files matching "report" + apps + other results	Open the file (if Files is global)

Matching is fuzzy: `fsrc` finds `File System (C:)`, `fmain.rs` finds `src/main.rs`.

Path browsing

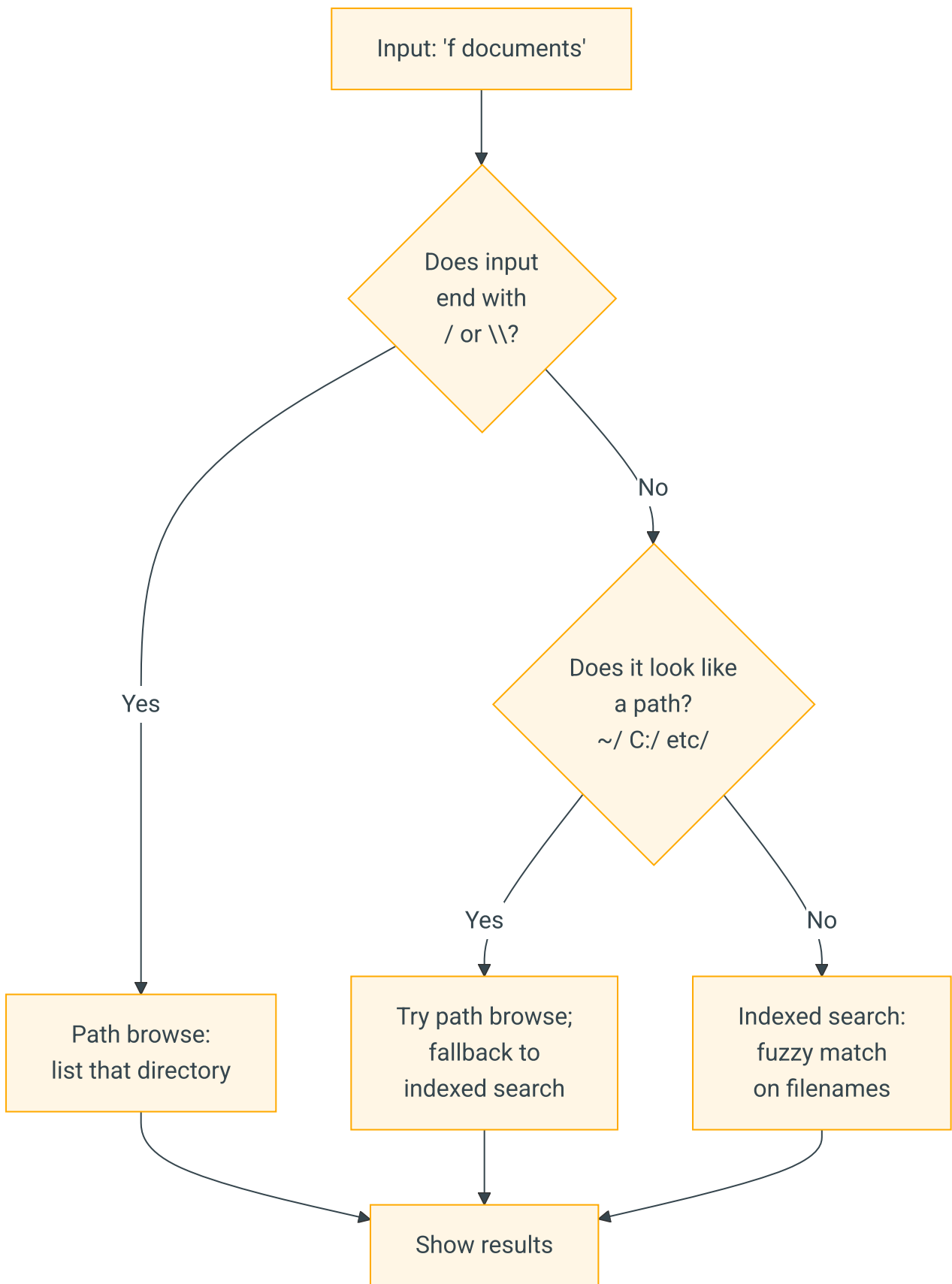
Type a path to browse its contents live. `Tab →` and `↑ Shift + Tab →` navigate up and down folder levels:

Input	What you see	Action
<code>~/Doc</code>	Contents of ~/Documents (path-matched)	Press <code>Tab →</code> to complete folder name
<code>~/Documents/</code>	Contents of Documents folder	<code>Enter ↵</code> on a folder to open it

Input	What you see	Action
<code>C:\Users\</code>	Contents of C:\Users	Navigate folders with <code>Tab →</code>
<code>/etc/</code>	Contents of /etc (Linux)	Press <code>↑ Shift</code> + <code>Tab →</code> to go up one level

Folders end with `/` (or `\` on Windows), so `Tab →` drills deeper. Path browsing shows contents live without waiting for the full index.

How indexing vs. path-browsing is chosen



Actions

Key(s)	Action
<code>Enter ↵</code>	Open the file or folder
<code>^ Ctrl</code> + <code>Enter ↵</code>	Show the file in your file manager (reveal in folder)
<code>⇧ Shift</code> + <code>Enter ↵</code>	Copy the full path to your clipboard
<code>Tab →</code> (path browsing)	Complete or drill into a folder
<code>⇧ Shift</code> + <code>Tab →</code> (path browsing)	Go up one folder level

Use `^ Ctrl` + `K` to see all available actions.

Options

Setting	Default	What it does	Config section
Directories	<code>~/Desktop</code> , <code>~/Documents</code> , <code>~/Downloads</code>	Folders whose contents are indexed	<code>[files]</code> <code>directories</code>
Max depth	<code>4</code>	How many folder levels below each directory are indexed	<code>[files]</code> <code>max_depth</code>
Include hidden	<code>false</code>	Whether to index dot-files and dot-folders (<code>.gitignore</code> , <code>.config/</code>)	<code>[files]</code> <code>include_hidden</code>
Keyword	<code>f</code>	Keyword to search only files	<code>[files]</code> <code>keyword</code>
Global	<code>true</code>	Also show file results in ordinary searches without the keyword	<code>[files]</code> <code>global</code>

How indexing works

At startup and every few minutes, Sevak scans your configured directories and builds an index. Results from indexed search come back instantly.

Pruned directories (always skipped to save time): - `node_modules`, `.git`, `target`, `__pycache__`, `.cache`, `venv`, `.venv`, `appdata`

To search inside one of these, use path browsing or configure them as a separate directory with a small depth.

Limits: The index is capped at 100,000 entries to bound memory. If you exceed this, older or less-used files drop out.

Platform notes

Windows

Paths use backslashes: `C:\Users\Name\Documents\`. Type `~` to expand to your home directory. Hidden files have the hidden attribute set; they appear in path browsing only if you explicitly type `.`.

macOS

Paths use forward slashes: `/Users/Name/Documents/`. The index includes system directories but respects macOS privacy settings (it cannot read files in `~/Library` without permission).

Linux

Paths use forward slashes: `/home/name/documents/`. The `~` shortcut expands to your home. Root directories like `/etc/` and `/usr/` are not automatically indexed; add them to `[files] directories` if you need to search them.

Tips and troubleshooting

Slow indexing: The first index scan takes longer than refreshes. It runs in the background; you can keep using Sevak while it builds.

Index doesn't include files I added: Refresh the index by choosing **Reload index** from the tray menu, or wait a few minutes for the automatic refresh.

Hidden files not showing: Path browsing respects `[files] include_hidden`; if it's false, hidden files appear only when the filter itself starts with a dot (e.g., `.git`).

Path completion is slow: Path browsing reads the filesystem live, so large directories (like `node_modules`) can take a moment. The indexed search is faster for large collections.

Symlinks: Sevak follows symlinks when indexing, so the same file can appear under different paths. If this is unwanted, add the symlinked directory to the pruned list in the code and recompile, or use path browsing to navigate directly.

Browser bookmarks

Search bookmarks from all your browsers and profiles. Everything is read locally from disk; nothing is sent anywhere. `Enter ↵` opens the bookmarked URL in your browser.

How to use it

Type `b` followed by a bookmark name or part of a URL:

Input	What you see	What <code>Enter ↵</code> does
<code>b github</code>	Bookmarks with "github" in the title or URL	Open github.com
<code>b documentation</code>	Pages bookmarked as "Documentation"	Open the URL
<code>b stackoverflow</code>	Stack Overflow bookmarks	Open the URL
<code>documentation</code> (no keyword)	Bookmarks matching "documentation" + apps + files	Open the bookmark (if global is enabled)

Sevak searches bookmark titles and URLs. Bookmarks are deduplicated across browsers and profiles; if the same URL exists in Chrome and Firefox, they merge into one result showing both browser names.

Supported browsers

Chromium family (Chrome, Edge, Brave, Vivaldi, Chromium, Opera, Opera GX on Windows and macOS): - Reads the `Bookmarks` JSON file from each profile - All profiles of each browser are searched automatically

Firefox family (Firefox, LibreWolf, Zen): - Reads `places.sqlite` from each profile - Profiles listed in `profiles.ini` are searched automatically

Actions

Key(s)	Action
Enter ↵	Open the bookmarked URL
⇧ Shift + Enter ↵	Copy the URL to your clipboard

Use `⌘ + K` to see all available actions. If a bookmark is stored in multiple browsers, the browser names appear in the result's subtitle.

Options

Setting	Default	What it does	Config section
Browsers	<code>[]</code> (auto-detect)	Which browsers to search (<code>[]</code> = all found, or <code>["chrome", "firefox"]</code>)	[bookmarks] browsers
Keyword	<code>b</code>	Keyword to search only bookmarks	[bookmarks] keyword
Global	<code>true</code>	Also show bookmarks in ordinary searches without the keyword	[bookmarks] global

Choosing which browsers to search

By default, Sevak searches every installed browser and all profiles. To limit it:

```
[bookmarks]
browsers = ["chrome", "firefox"]    # search Chrome and Firefox only
browsers = []                      # search all browsers (default)
browsers = ["edge"]                # search Edge only
```

Valid browser ids: `chrome`, `edge`, `brave`, `vivaldi`, `chromium`, `opera`, `opera-gx`, `firefox`, `librewolf`, `zen`.

How indexing works

Bookmarks are indexed at startup and every few minutes in the background. Sevak reads each browser's bookmarks file only when it changes (checked by file modification time), so an idle refresh is cheap.

Limits: The index is capped at 50,000 bookmarks to bound memory. If you exceed this, older bookmarks may not appear.

Excluded: Bookmarks whose URL is not `http(s)` (e.g., `javascript:`, `chrome:`, `file:`, `place:`) are skipped.

Platform notes

Windows

Browsers store bookmarks in `%APPDATA%\<browser>\User Data\Default\Bookmarks` (Chrome family) or `%APPDATA%\<browser>\Profiles\<profile>\places.sqlite` (Firefox). Sevak reads from these locations automatically.

macOS

Bookmarks are stored in `~/Library/Application Support/<browser>/`. Sevak respects system privacy settings and may ask for permission to read the Library folder.

Linux

Bookmarks are stored in `~/.config/<browser>/` or `~/.mozilla/firefox/`. Sevak reads from these locations automatically.

Tips and troubleshooting

Bookmarks don't appear: Open **Settings** and check that bookmarks are enabled under **Plugins**. If the browser was just installed, choose **Reload index** from the tray menu.

Browser not detected: Sevak looks for browsers in their default locations. If you installed a browser somewhere unusual, it may not be found. In that case, bookmark a page in your default browser or use path browsing to navigate to the bookmark file manually.

Too many bookmarks: If you have 50,000+ bookmarks (rare!), the oldest or least-used ones may not be indexed. Reduce the number of bookmarks or add more of your favorites to the top level of your bookmark structure so they stay in the index.

Merging profiles: If you use multiple profiles in the same browser, Sevak searches all of them. Bookmarks with the same URL appear as one result showing both profile names in the subtitle.

Bookmarks not updating: The index refreshes every few minutes. If you just added a bookmark, wait a moment or choose **Reload index** from the tray menu to see it immediately.

Clipboard history

Store and search text you've copied recently. Type `cb <text>` to find a clipboard entry and paste it back into the app you were using. Entries are stored newest first, and duplicates are merged (the oldest copy disappears, the newest stays).

This feature is **opt-in**; nothing is watched or stored unless you turn it on.

Turn it on

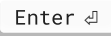
1. Open **Settings** (tray or `sevak --settings`), go to **Plugins**, and enable **Clipboard history**.
2. Or manually edit `config.toml` and set `[clipboard] enabled = true`, then reload.

The first time you enable it, Sevak starts watching the clipboard. Everything copied *after* that moment is remembered; what was on the clipboard before is not.

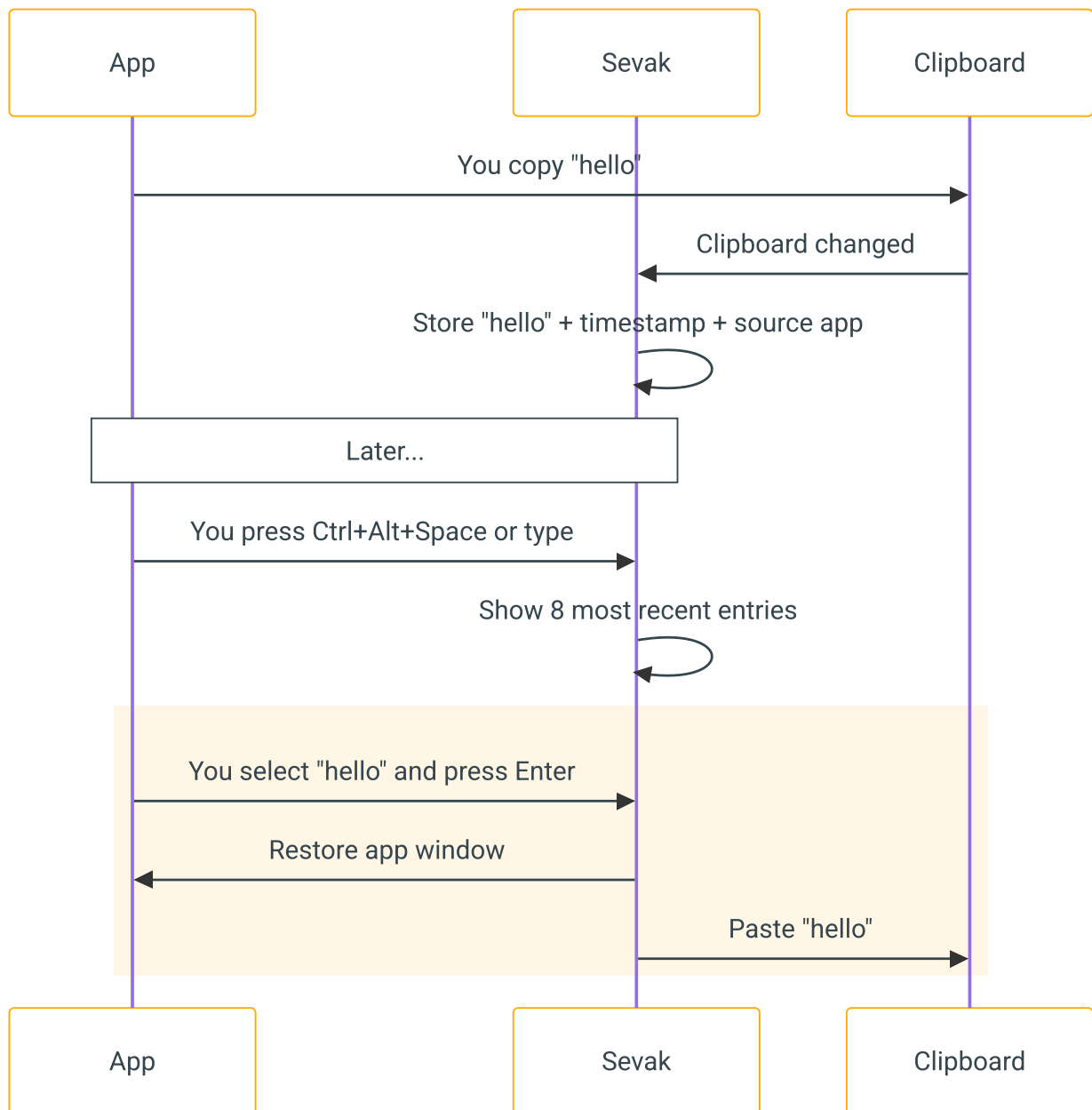
How to use it

Type `cb` followed by at least one character of the text you want to find:

Type	Shown	Press 
<code>cb test</code>	Clipboard entries containing "test", newest first	Pastes the selected entry into the app you were in
<code>cb clear</code>	A "Clear clipboard history" row	Permanently deletes all clipboard history

When you press , Sevak hides, brings back the app that was in focus before you opened Sevak, and pastes the text (unless [pasting is unavailable](#)).

Example



Actions

- **Enter** : Paste the selected entry into the app you were in.
- **^ Ctrl + K** (action panel): See all available actions for the selected entry (typically just paste and copy).
- **^ Ctrl + C** : Copy the selected entry to the clipboard without pasting into the previous app.

Options

Setting	Default	What it does	Config section
Enabled	<code>false</code>	Turn clipboard history on or off	<code>[clipboard]</code> <code>enabled</code>
Max items	<code>200</code>	Keep this many entries; oldest are dropped (1–5000)	<code>[clipboard]</code> <code>max_items</code>
Max item size	<code>64 KB</code>	Longer text is not recorded (1 B–4 MB)	<code>[clipboard]</code> <code>max_item_bytes</code>
Ignore apps	<code>(empty)</code>	Never record copies made in these apps (case-insensitive names), e.g. <code>["KeePassXC", "1Password"]</code>	<code>[clipboard]</code> <code>ignore_apps</code>
Restore clipboard	<code>false</code>	Put back the clipboard's previous text after pasting	<code>[paste]</code> <code>restore_clipboard</code>

What is and isn't recorded

Recorded: - Plain text copies (only text, not images, files or rich content) - The timestamp (seconds since the Unix epoch) - The name of the app where it was copied from (where available)

Never recorded: - Content marked secret by the app (password managers on Windows and macOS set a marker; Linux has no marker, so use `ignore_apps`) - Copies made while one of the `ignore_apps` had focus - Text longer than `max_item_bytes` - Empty or whitespace-only text - Copies Sevak itself made (pastes, or copies from its own actions) - Images, files, rich text, HTML or any non-text content

The clipboard you started with: - When you enable clipboard history, it does not record what was already on the clipboard—only what you copy *after* enabling it.

Storage and privacy

- **Location:** `clipboard-history.json` in Sevak's data folder (see [Files and data](#) for the path by platform).
- **Permissions:** On Linux and macOS, the file is readable only by your user (mode `0600`). On Windows, it inherits NTFS permissions from its folder.
- **Clearing:** Type `cb clear` (any start of "clear clipboard history", at least 3 letters) to show a **Clear clipboard history** row, then press `Enter` on it to delete the entire history. There is no further prompt, and it cannot be

undone.

- **On uninstall:** The `clipboard-history.json` file stays behind. Delete it manually if you want to remove the history.

Pasting: platform notes

Windows

Pasting works in most applications. **Exception:** applications running with elevated privileges (Admin, UAC elevation) cannot receive simulated keypresses, so Sevak copies the entry instead of pasting and the row says so.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → Sevak to paste. Without this permission, Sevak copies instead and the row shows "Copies to clipboard". Keyboard layouts that move the `v` key (e.g. Dvorak) also prevent pasting; use copy instead.

Linux (X11)

Pasting works in most applications.

Linux (Wayland)

Applications cannot simulate keypresses, so Sevak can only copy the entry. The row shows "Copies to clipboard". If you need pasting on Wayland, set `[actions] use_clipboard_fallback = true`, copy the entry into the clipboard yourself, and use Universal Actions (Ctrl+Alt+Space) to apply transformations.

Tips and troubleshooting

Password managers

If you use KeePassXC, 1Password, Bitwarden or another password manager, add its name to `[clipboard] ignore_apps` to prevent passwords and secret keys from being recorded. Sevak already skips content marked secret by the app (Windows and macOS), but Linux has no such marker.

Restore the clipboard

If you want pasting to restore the clipboard's previous content (so `cb <text>` does not permanently replace what you had copied), set `[paste] restore_clipboard = true`. This is useful when the text you're pasting is temporary and you want to keep your existing clipboard.



Limit the history size

Each entry takes a little disk space. If you have many very long entries (near the `max_item_bytes` limit), the history file can grow. Increase `max_items` to trim entries more aggressively, or lower `max_item_bytes` to skip long text.



History is not encrypted

The `clipboard-history.json` file contains the text you copied, in plain text. If your computer is stolen or your home folder is otherwise compromised, the history can be read. Only enable clipboard history if you are comfortable with this risk, or keep sensitive copies out of it by using `ignore_apps`.



App names must be exact (or close)

For `ignore_apps`, Sevak matches the application or window name case-insensitively. For example, `KeePassXC` and `keepassxc` both work, but `KeePass` (without `XC`) might not. If ignoring is not working, check the actual name of the app in the clipboard entry's subtitle (e.g. "Copied from KeePassXC").

Snippets

Paste text you define: boilerplate, email signatures, code templates, or anything you type often. Type `s <name>` to find a snippet and paste it with placeholders filled in.

How to use it

Define snippets in your `config.toml`:

```
[[snippet]]
name = "Email signature"
keyword = "sig"
text = "Best regards, \nNinad\n{date}"

[[snippet]]
name = "License header"
keyword = "lic"
text = "# Apache License 2.0 - https://example.com"

[[snippet]]
name = "Meeting notes"
text = ""
## Attendees
-

## Agenda
-

## Action items
-
""
```

Then search by name or keyword:

Type	Shown	Press Enter ↵
<code>s email</code>	"Email signature" (matches name)	Pastes the text with <code>{date}</code> replaced by today
<code>s sig</code>	"Email signature" (matches keyword)	Pastes the text
<code>s lic</code>	"License header" (exact keyword match ranked highest)	Pastes the text

Type	Shown	Press <code>Enter ↵</code>
<code>s meet</code>	"Meeting notes" (fuzzy match on name)	Pastes the text (no placeholders to fill)

When you press `Enter ↵`, Sevak hides, brings back the app that was in focus, and pastes the snippet (unless [pasting is unavailable](#)).

Keyword matching

A `keyword` is optional and gives the snippet an alternative search term:

- Type `s signature` → finds "Email signature" by name.
- Type `s sig` → finds "Email signature" by keyword (and ranks higher).
- Type `s email sig` → finds "Email signature" because it matches either term.

An exact match on a keyword outranks any fuzzy match on the name, so `s sig` shows "Email signature" first even if you have other snippets with "sig" in the name.

Placeholders

Placeholders are **filled in when you press** `Enter ↵`, not while you type. This means `{time}` gives the time of the paste, and `{clipboard}` reads the clipboard only then.

Placeholder	Result	Example
<code>{date}</code>	Today in <code>YYYY-MM-DD</code> format	<code>2026-10-03</code>
<code>{time}</code>	Now in <code>HH:MM</code> format (24-hour)	<code>14:05</code>
<code>{datetime}</code>	Today and now together	<code>2026-10-03 14:05</code>
<code>{date:FORMAT}</code>	Today in <code>strftime</code> format	<code>{date:%d %B %Y}</code> → <code>03 October 2026</code>
<code>{time:FORMAT}</code>	Now in <code>strftime</code> format	<code>{time:%H:%M:%S}</code> → <code>14:05:23</code>

Placeholder	Result	Example
<code>{datetime:FORMAT}</code>	Now in strftime format	<code>{datetime:%Y-%m-%d at %H:%M}</code> → 2026-10-03 at 14:05
<code>{clipboard}</code>	The text on the clipboard when you pressed <code>Enter ↵</code>	(whatever you had copied)
<code>{uuid}</code>	A fresh random UUID v4	550e8400-e29b-41d4-a716- 446655440000
<code>{{</code>	A literal <code>{</code>	<code>{{key}}</code> → <code>{key}</code>
<code>}}</code>	A literal <code>}</code>	(same)

Anything else in braces (`{unknown}` , `{{}` , a lone `{ }`) is left exactly as written, so code snippets with braces usually just work:

```
[[snippet]]
name = "Python lambda"
text = "lambda x: {x * 2}" # The {x * 2} is left as-is
```

Common formats

A few handy strftime formats for custom dates and times:

```
[[snippet]]
name = "ISO week date"
text = "{date:%G-W%V}" # 2026-W40

[[snippet]]
name = "US date"
text = "{date:%m/%d/%Y}" # 10/03/2026

[[snippet]]
name = "File timestamp"
text = "backup_{datetime:%Y%m%d_%H%M%S}" # backup_20261003_140523

[[snippet]]
name = "Markdown heading + date"
text = "## {date:%A, %B %-d, %Y}" # ## Thursday, October 3, 2026
```

Actions

- **Enter** : Paste the selected snippet into the app you were in.
- **^ Ctrl** + **K** (action panel): See all available actions for the selected snippet (typically just paste and copy).
- **^ Ctrl** + **C**: Copy the selected snippet's text to the clipboard without pasting into the previous app. Placeholders are not filled.

Options

Setting	Default	What it does	Config section
Snippets	<i>(empty)</i>	List of <code>[[snippet]]</code> entries with name, optional keyword, and text	<code>[[snippet]]</code>
Restore clipboard	<code>false</code>	Put back the clipboard's previous text after pasting (also affects clipboard history)	<code>[paste]</code> <code>restore_clipboard</code>

Pasting: platform notes

Windows

Pasting works in most applications. **Exception:** applications running with elevated privileges (Admin, UAC elevation) cannot receive simulated keypresses, so Sevak copies instead and the row says so.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → Sevak to paste. Without this permission, Sevak copies instead and the row shows "Copies to clipboard". Keyboard layouts that move the  key (e.g. Dvorak) also prevent pasting; use copy instead.

Linux (X11)

Pasting works in most applications.

Linux (Wayland)

Applications cannot simulate keypresses, so Sevak can only copy. The row shows "Copies to clipboard".

Editing snippets

The **Settings** window does not let you edit snippets (to keep hand-written entries untouched, with proper formatting and comments). Edit them by opening `config.toml` directly:

1. Choose **Settings** → **Open config file** (or find it in [Files and data](#)).
2. Edit the `[[snippet]]` sections.
3. Save the file and choose **Reload index** from Sevak's tray or restart.

Each `[[snippet]]` must have a `name` and `text`; the `keyword` is optional. Names do not have to be unique (if you have duplicates, Sevak adds a number like "Email #2" to each).

Tips and troubleshooting



Shared snippets

Store `config.toml` in a synced folder (Dropbox, OneDrive, iCloud Drive, git) with `SEVAK_CONFIG_DIR=~/.Dropbox/sevak`. The `[[snippet]]` entries sync across devices. See [Configuration reference](#).



Long multiline snippets

Use triple-quoted TOML strings to avoid escaping newlines:

```
[[snippet]]
name = "Meeting notes"
text = """
## Attendees
-

## Agenda
-
"""
```



Generate UUIDs

Each time you paste a snippet with `{uuid}`, a fresh UUID is generated. Use it in multiple places in a snippet to get different UUIDs:

```
[[snippet]]
name = "Pair of UUIDs"
text = "{uuid}\n{uuid}"
```





Escaping braces in JSON or code

Snippets containing braces (JSON, programming languages) can conflict with placeholders. Use `{{` and `}}` to escape them:

```
[[snippet]]
name = "JSON template"
text = "{{\"user\": \"{clipboard}\"}}"
# Pastes: {"user": "..."}

```



System commands

Lock your screen, put the computer to sleep, restart, shut down, and more. Type a command's name as you would an app name (two or more characters); only the commands available on your machine are shown.

How to use it

Type the command name (or an alias):

Command	Aliases	What it does
lock	lock, lock computer, lock workstation	Lock the screen or session
sleep	suspend, standby	Put the computer to sleep
hibernate	(no aliases)	Hibernate (if supported)
restart	reboot	Restart the computer
shut down	shutdown, power off, poweroff, turn off	Power off the computer
log out	logout, log off, logoff, sign out, signout	End your session
empty trash / recycle bin	empty recycle bin, clear trash, recycle bin	Delete all trash (or Recycle Bin on Windows)

Press `Enter` to run the command. Destructive commands (restart, shut down, log out, empty trash) ask for confirmation first, unless you turn it off.

Aliases

Each command has aliases so you can type less. A query of at least two characters finds these commands (one letter alone might match app names like "S" for Slack, so Sevak requires more). Examples:

Query	Found
lo	Lock screen, Log out
sl	Sleep
sh	Shut down
re	Restart

System settings

Sevak also offers shortcuts to common operating system settings pages. Type the page name:

Page	Available on	Example queries
Bluetooth	Windows, macOS, Linux	bluetooth
Display	Windows, macOS, Linux	display, screen
Wi-Fi	Windows, macOS, Linux	wifi, network
Sound	Windows, macOS, Linux	sound, audio
Network	Linux	network
Power	Windows, Linux	power, battery

Actions

- **Enter** : Run the command (with confirmation if configured).
- **^ Ctrl + K** (action panel): See all available actions (typically just run).

Options

Setting	Default	What it does	Config section
Confirm	<code>true</code>	Ask before restart, shut down, log out, and empty trash	<code>[system]</code> <code>confirm</code>
Disabled	<i>(empty)</i>	Hide specific commands or settings pages: <code>"restart"</code> , <code>"settings:bluetooth"</code> , etc.	<code>[system]</code> <code>disabled</code>

Platform notes

Windows

- **Lock:** `LockWorkStation` (fast)
- **Sleep:** `SetSuspendState` (hybrid sleep mode)
- **Hibernate:** `shutdown /h` (only if hibernation is enabled)
- **Restart:** `shutdown /r /t 0`
- **Shut down:** `shutdown /s /t 0`
- **Log out:** `shutdown /l`
- **Empty Recycle Bin:** `SHEmptyRecycleBin` (permanent delete)
- **Settings pages:** `ms-settings:` (Bluetooth, Display, Network, Power, Sound, etc.)

macOS

- **Lock:** `pmset displaysleepnow` (lock screen; requires "Require password" set to immediately)
- **Sleep:** `pmset sleepnow`
- **Hibernate:** Not offered (use sleep instead)
- **Restart:** Uses System Events (requires Accessibility permission on first use)
- **Shut down:** Uses System Events (requires Accessibility permission on first use)
- **Log out:** Uses System Events (requires Accessibility permission on first use)
- **Empty Trash:** Finder (requires Accessibility permission on first use)
- **Settings pages:** System Settings panes

Linux

- **Lock:** `loginctl lock-session`
- **Sleep:** `systemctl suspend`
- **Hibernate:** `systemctl hibernate` (only if swap is set up)
- **Restart:** `systemctl reboot`
- **Shut down:** `systemctl poweroff`
- **Log out:** `gnome-session-quit` (GNOME), or KDE `qdbus`, or `loginctl terminate-session`
- **Empty Trash:** `gio trash --empty`
- **Settings pages:** `gnome-control-center <panel>` (if installed; GNOME only)

Tips and troubleshooting



Disable a command

To hide a specific command, add it to `[system] disabled`:

```
[system]
disabled = ["hibernate", "settings:bluetooth"]
```



Disable all confirmations

If you do not want to confirm destructive commands, set `confirm = false`:

```
[system]
confirm = false
```



Destructive commands ask for confirmation

Restart, shut down, log out, and empty trash are destructive and ask for confirmation first (if `confirm = true`). A warning dialog appears whose button names the action (for example **Restart**). Only that button runs it; **Cancel** or closing the dialog does nothing. This also applies when the command is bound to a [custom hotkey](#).



macOS Accessibility permission

Restart, shut down, log out, and empty trash on macOS need *System Settings* → *Privacy & Security* → *Accessibility* → *Sevak*. The first time you use any of these, macOS shows a permission prompt; grant it and try again.

Shell commands

Run shell commands in a terminal window. Type `> <command>` (or `><command>` , no space needed); nothing happens until you press `Enter ↵`. Recent commands are offered again.

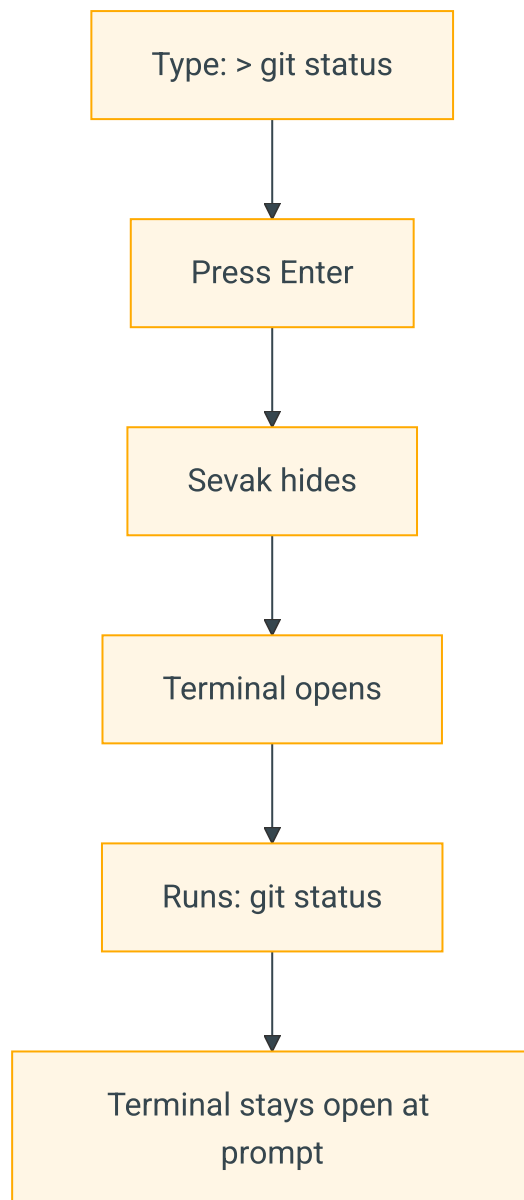
How to use it

Type `>` followed by the command:

Type	Shown	Press <code>Enter ↵</code>
<code>> git status</code>	"Run <code>git status</code> in terminal"	Opens your terminal and runs the command
<code>>npm test</code>	"Run <code>npm test</code> in terminal"	Opens your terminal and runs the command
<code>></code> (just <code>></code>)	"Open terminal" + recent commands	Opens a terminal at a shell prompt (if a recent is selected, runs it)

Nothing is executed until you press `Enter ↵`. Recent commands are stored in `usage.json` and are offered again, newest first and ranked by frequency.

Example



Keep the terminal open

By default, after your command finishes, the terminal stays open at a shell prompt (`[shell] keep_open = true`). If you want it to close when the command exits, set `keep_open = false` .

Choosing a terminal and shell

Sevak auto-detects a suitable terminal and shell for your platform, but you can specify one manually:

```
[shell]
terminal = ""          # "" = auto-detect
shell = ""             # "" = auto-detect
keep_open = true       # stay at a prompt after the command exits
```



Auto-detection order

Windows

Terminal: Windows Terminal (`wt`), else a console window.

Shell: `pwsh` (PowerShell Core), else `powershell`, else `cmd`.

For PowerShell, the command is encoded as Base64 to avoid quoting issues.

macOS

Terminal: Terminal.app (or `iTerm2` if you set `terminal = "iterm"`, `terminal = "iterm2"`, or `terminal = "/Applications/iTerm.app"`).

Shell: Your login shell (environment variable `$SHELL`), run by the terminal. Sevak does not choose the shell on macOS.

Linux

Terminal: `$TERMINAL` environment variable, then `x-terminal-emulator`, `gnome-terminal`, `konsole`, `kitty`, `alacritty`, `wezterm`, `foot`, `xterm`.

Shell: `$SHELL`, else `sh`.

Examples

Choose a specific terminal or emulator:

```
[shell]
# Use Windows Terminal explicitly
terminal = "wt"

# Or an alternative like ConHost (Windows console)
terminal = "conhost"

# macOS: use iTerm2 instead of Terminal.app
terminal = "iterm"

# Linux: use Kitty with custom arguments
terminal = "kitty --class sevak"

# Linux: use a terminal by full path
terminal = "/usr/bin/alacritty"
```

Choose a different shell:

```
[shell]
# Use bash on Windows (with Git Bash)
shell = "bash"

# Use zsh on Linux
shell = "zsh"

# PowerShell on Linux
shell = "pwsh"
```



Actions

- **Enter** : Run the command in a terminal window.
- **^ Ctrl** + **K** (action panel): See all available actions (typically just run).

Options

Setting	Default	What it does	Config section
Terminal	<i>(auto-detect)</i>	Terminal program: <code>wt</code> , <code>iterm</code> , <code>kitty</code> , <code>alacritty</code> , or a full path with optional args	[shell] terminal
Shell	<i>(auto-detect)</i>	Shell that runs the command: <code>pwsh</code> , <code>bash</code> , <code>zsh</code> , <code>fish</code> , etc.	[shell] shell
Keep open	<code>true</code>	Leave the terminal at a shell prompt after the command exits	[shell] keep_open

Platform notes

Windows

Commands run in the terminal without a shell context, so shell aliases and functions (from `.bashrc`, `.zshrc`, etc.) are not available. Use full paths or install executables on `PATH`. PowerShell scripts need `powershell - ExecutionPolicy Bypass -File <path> .`

macOS

Commands run through your login shell, so aliases and functions from your shell profile are available.

Linux

Commands run through `$SHELL` (or `sh`) in non-interactive mode, so aliases from `.bashrc` (and similar) are typically not available. Use `source ~/.bashrc && <command>` if you need aliases or custom functions.

Tips and troubleshooting



Keep recent commands

Recent commands are stored in `usage.json` (in Sevak's data folder). Clearing the usage store deletes them. Choose **Reload index** from the tray to rebuild, or restart Sevak.



Set a variable for the terminal

Some terminal programs recognize environment variables. For example, on Linux you can set `KITTY_LISTEN_ON` to enable remote scripting, or use `alacritty --class sevak` to apply different window decoration.



Elevated commands on Windows

Commands run as the current user. If you need to run a command as administrator, use `runas /user:Administrator <command>` instead (this will prompt for your admin password).



Terminal is missing

If Sevak cannot find a terminal on Linux, install one from the auto-detection list, or set `terminal` to a full path. Check that the terminal is on `PATH`: `which alacritty` or `which gnome-terminal`.



Shell not found

If the shell you specified is not installed, Sevak falls back to `sh` (Linux) or tries the next option in the auto-detect order (Windows). Verify the shell exists: `which bash` or `which pwsh`.

Selection actions

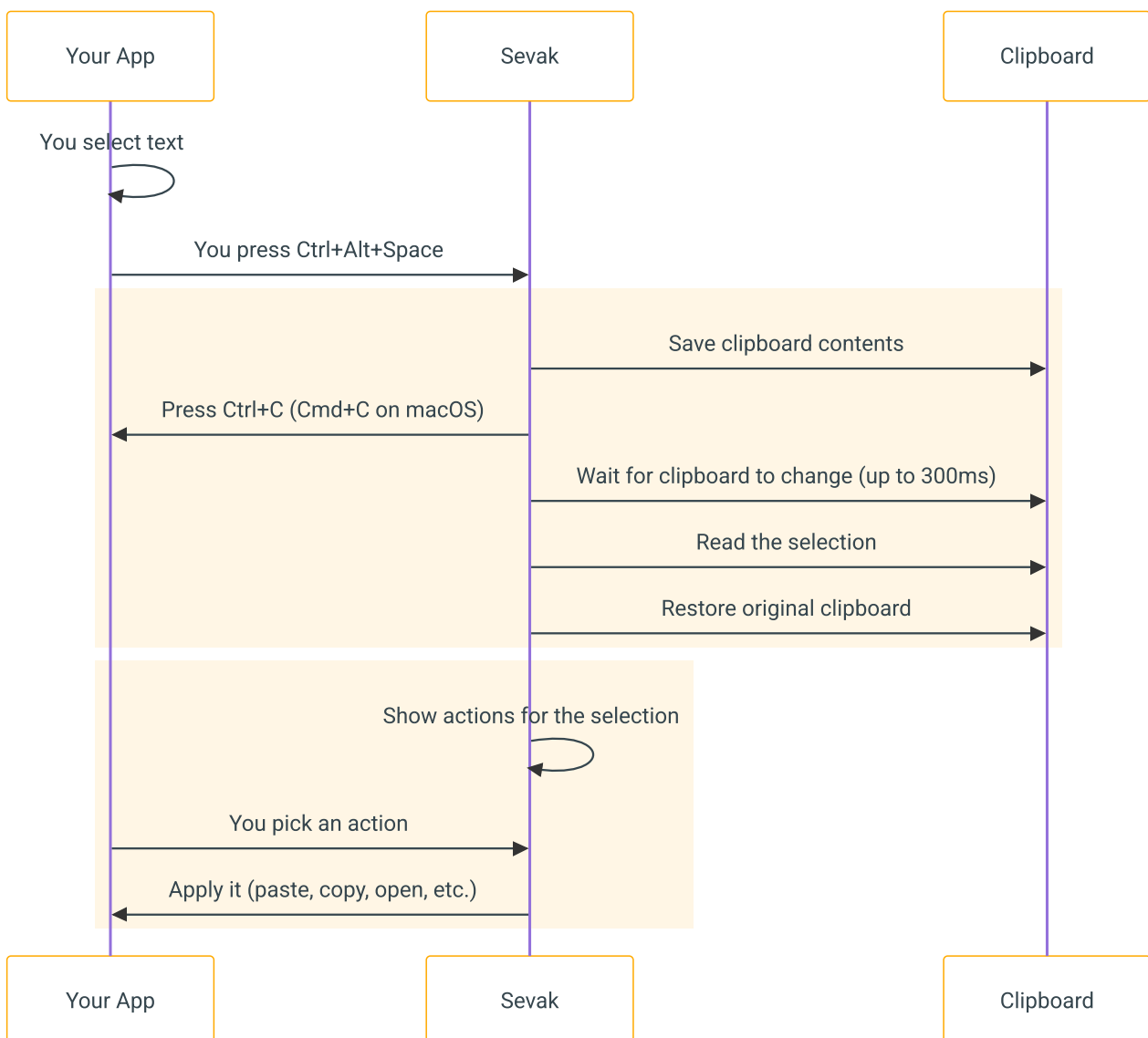
Act on text, links, or files selected in another app without copying and pasting manually. Select something, press `Ctrl+Alt+Space` (configurable), and Sevak shows actions for it. Nothing runs until you pick an action with `↑ Up` / `↓ Down` and `Enter ↵`, or `Ctrl+1` through `Ctrl+9`.

Turn it on

The feature is on by default. To change the hotkey or turn it off:

```
[general]
actions_hotkey = "Ctrl+Alt+Space" # "" turns it off
```

How it works



The selection itself is **never stored or logged**. Sevak reads it once, shows actions, and forgets it unless you run an action.

What you can do with selections

Text

Select any text:

Action	What it does
Search [Engine] for "..."	Opens your browser with a web search (one row per <code>[[web_search]]</code> engine)
Large Type	Shows the text huge across the screen (for reading across the room)
Copy	Copies the text to the clipboard
Paste as plain text	Pastes without formatting (font, color, etc.)
Calculate	If the text is a calculation or conversion, shows the result (e.g., <code>2*3+4</code> , <code>10 km in mi</code>)
Uppercase	Transform: <code>hello</code> → <code>HELLO</code> (replaces the selection where pasting works)
Lowercase	Transform: <code>HELLO</code> → <code>hello</code>
Title Case	Transform: <code>hello world</code> → <code>Hello World</code>
Trim whitespace	Remove leading and trailing spaces
URL-encode	Transform: <code>hello world</code> → <code>hello%20world</code>
URL-decode	Transform: <code>hello%20world</code> → <code>hello world</code>
Base64 encode	Transform: <code>hello</code> → <code>aGVsbG8=</code>
Base64 decode	Transform: <code>aGVsbG8=</code> → <code>hello</code> (only if valid)

Action	What it does
Pretty-print JSON	Format minified JSON with indentation
Minify JSON	Remove whitespace from JSON

Transformations **replace the selection** in the app (Sevak pastes over it) where pasting works. `Ctrl+Enter` copies the result instead of pasting. If pasting is not available (Wayland, or macOS without permission), they copy.

URLs and links

Select one or more URLs (starting with `http://`, `https://`, `mailto:` or `www.`):

Action	What it does
Open	Opens the URL(s) in your default browser
Copy	Copies to the clipboard (one line per URL)
Large Type	Shows the URL(s) huge on screen

Files and folders

Select file paths or drag files from a file manager:

Action	What it does
Open	Opens the file with its default application, or a folder in the file manager
Show in folder	Opens the file manager at the file's location
Copy path	Copies the full path to the clipboard (one line per file)
Open in terminal	Opens a terminal in the folder (folders only)
Run as administrator	Runs the program with elevated privileges (Windows .exe programs only)

Action	What it does
Send to Sevak	Fills Sevak's search box with the path for browsing (path browsing)

Text that looks like a path

If you select text that matches a file path (`C:\Users\me\Documents` , `/home/me/Projects` , `~/Downloads`), Sevak also offers the file actions above, then the text actions (search, calculate, etc.).

Actions and modifiers

Press `Enter ↵` to run the highlighted action. Modifiers change the behavior:

Key	Effect	Example
<code>Enter ↵</code>	Run the action (as shown)	Paste the transformation, open the URL, etc.
<code>^ Ctrl</code> + <code>Enter ↵</code>	Copy instead of pasting (transformations only)	Copy <code>HELLO</code> to clipboard instead of pasting
<code>↑ Shift</code> + <code>Enter ↵</code>	Copy the URL (web searches only)	Copy the search URL without opening it

The row under each action hints at the modifiers.

Pick an action with number keys

Instead of arrow keys and Enter, use `Ctrl+1` through `Ctrl+9` to run the 1st through 9th action directly:

Key	Effect
<code>^ Ctrl</code> + <code>1</code>	Run the 1st action
<code>^ Ctrl</code> + <code>2</code>	Run the 2nd action

Key	Effect
...	...
  	Run the 9th action

Dismiss the action panel

-   : Close the action panel without running anything.
-   : Close the action panel (same as Esc).
- Any other key: Close and continue typing in Sevak.

Platform notes

Windows

Selection capture works in every app **except windows running with elevated privileges** (Admin, UAC). Sevak tries anyway and shows what happens. For admin windows, select manually and set `[actions] use_clipboard_fallback = true` to act on what you copied.

macOS

Requires *System Settings* → *Privacy & Security* → *Accessibility* → *Sevak*. Without this permission, Sevak cannot press Cmd+C in other apps and shows "Cannot read selection". Grant the permission and try again.

Files in Finder and text both work. Pasting transformations also needs Accessibility permission and respects your keyboard layout.

Linux (X11)

By default, Sevak reads the **PRIMARY selection** (text you just highlighted) first, without pressing any key (configurable: `[actions] use_primary_selection = true`). This is instant. If nothing is highlighted, it falls back to Ctrl+C.

You can also turn off primary selection and always use Ctrl+C: `use_primary_selection = false`.

Files are read from the clipboard. Terminal windows are skipped (Ctrl+C would interrupt the running program); use manual selection and clipboard fallback instead.

Linux (Wayland)

Applications cannot read another app's selection or press keys. Sevak shows "Cannot read selection on this desktop" or similar. Workaround:

1. Select the text manually.
2. Copy it (Ctrl+C or Cmd+C).
3. Set `[actions] use_clipboard_fallback = true` in your config.
4. Press Ctrl+Alt+Space again.

Sevak will then act on what you copied. This is not automatic, but it lets you use transformations and web searches.

Special cases

Very large selections

Sevak refuses selections larger than 256 KB to avoid lag. If you have a massive block of text selected, copy a smaller part and try again.

Terminal windows

On Windows and Linux, terminal windows (Windows Terminal, `cmd`, PowerShell, `gnome-terminal`, `konsole`, etc.) do not receive Ctrl+C from Sevak because it would interrupt the running program. Select manually or use clipboard fallback instead.

No admin permission needed on macOS

Unlike pasting, selection capture on macOS needs the same Accessibility permission but does not require you to run Sevak as admin.

Configuration

```
[general]
actions_hotkey = "Ctrl+Alt+Space"      # Hotkey to trigger actions; "" turns it off

[actions]
use_primary_selection = true           # Linux X11: read highlighted text first
use_clipboard_fallback = false         # Act on clipboard when capture fails
```

See [Configuration reference](#) for details on all settings.

Tips and troubleshooting

Bind actions to a different key

Choose a key not used by your desktop. On some layouts, Ctrl+Alt+Space types a no-break space; if that is your case, change the hotkey to `Ctrl+Alt+C` or another combination.

Use clipboard fallback on Wayland

If you are on Wayland and want to use transformations without the selection capture workaround, set `use_clipboard_fallback = true`. Then copy what you want to transform, press the hotkey, and pick an action. The result pastes or copies as normal.



Selection capture blocks keyboard input briefly

When you press the hotkey, Sevak waits for the modifiers to be released (up to 600ms) to avoid sending Ctrl+C while Ctrl is still down. If the hotkey's modifiers are held too long, Sevak releases them for the app. This ensures the copy succeeds, but it means your key bindings using those modifiers might behave unexpectedly for a moment.

Script plugins

Extend Sevak with custom search results by writing a script in Python, Node.js, PowerShell, or any language. A script plugin is a folder with a manifest (`plugin.toml`) and a script; drop it into your plugins folder and Sevak loads it on the next startup (or after **Reload index**).

This page covers installing and using script plugins. For writing one, see [Writing plugins](#).

Install a plugin

1. **Get the plugin folder.** Download or clone a script plugin (examples are in the [Sevak repository](#)).
2. **Find your plugins folder:**
3. **Windows:** `%APPDATA%\sevak\plugins`
4. **macOS:** `~/Library/Application Support/sevak/plugins`
5. **Linux:** `~/.config/sevak/plugins`

Or in Sevak, choose **Settings** → **Open config file**, then go up one folder to find the plugins folder.

1. **Copy the plugin folder** (with its `plugin.toml` and script files) into the plugins folder. The folder structure looks like:

```
~/.config/sevak/  
├─ config.toml  
├─ plugins/  
│   ├─ hello-python/  
│   │   ├─ plugin.toml  
│   │   └─ main.py  
│   └─ timestamp-powershell/  
│       ├─ plugin.toml  
│       └─ timestamp.ps1
```

1. **Approve the plugin** (first time only):
2. Restart Sevak or choose **Reload index** from the tray menu.
3. Sevak finds the new plugin and shows a warning dialog, **Sevak: new script plugin**, with its name, keyword, folder and the exact command it runs.
4. Click **Allow** only if you trust where the plugin came from: it runs with your account's permissions. **Not now** leaves it off and asks again the next time Sevak starts.
5. Sevak remembers the answer per plugin and command. If a plugin's command changes, you are asked again.

The plugin is now active. Type its keyword (defined in `plugin.toml`) to search it.

How to use a script plugin

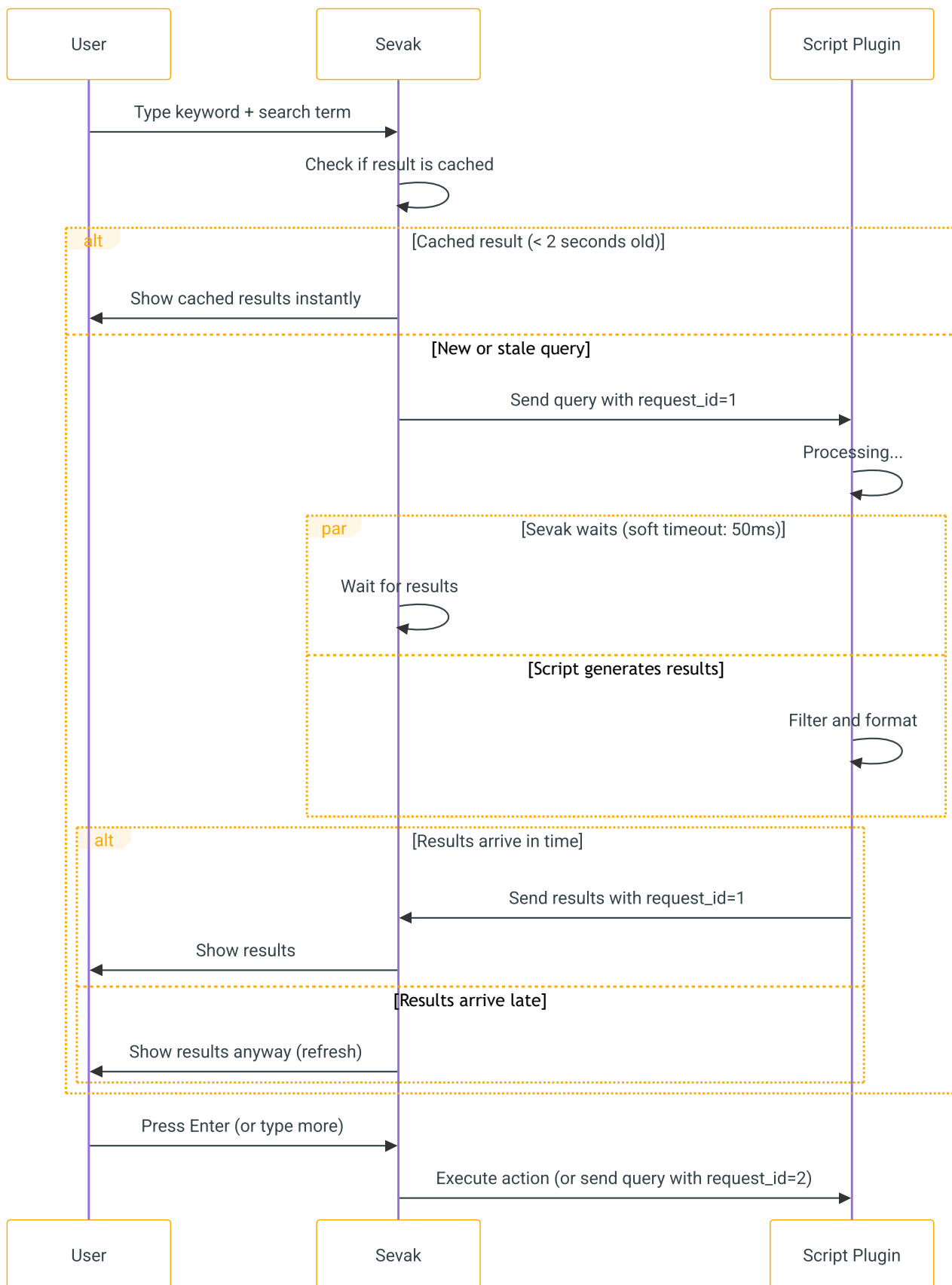
Each script plugin has a **keyword** (defined in its `plugin.toml`). Type the keyword followed by your search term:

Type	Shown	Press Enter ↵
<code>hello john</code>	Results from the "Hello" plugin matching "john"	Runs the action for the selected result
<code>ts</code>	Results from the "Timestamp" plugin (if ts is the keyword)	Runs the action

Results come from the script's output; the script decides what actions do (copy, paste, open a URL, etc.).

How script plugins communicate

A persistent-mode script runs once and listens on stdin for queries. Sevak and the script speak JSON lines:



Example: built-in plugins

Sevak's built-in plugins work the same way:

- `cb` is clipboard history's keyword
- `s` is snippets' keyword
- `>` is shell commands' keyword
- `f` is file search's keyword

Script plugins work exactly like these: type the keyword, get results, press Enter.

Built-in examples

The Sevak repository includes example plugins you can install:

hello-python

A Python plugin that greets you. Demonstrates a persistent process answering queries on stdin/stdout.

- **Location:** `examples/plugins/hello-python/`
- **Keyword:** `hello`
- **How to use:** Type `hello john` to see a greeting.

timestamp-powershell

A PowerShell plugin that generates timestamps. Demonstrates using PowerShell on Windows and other platforms.

- **Location:** `examples/plugins/timestamp-powershell/`
- **Keyword:** `ts`
- **How to use:** Type `ts` to see timestamp options (now, yesterday, tomorrow, etc.).

case-converter-node

A Node.js plugin that converts text case (uppercase, lowercase, title case, etc.). Demonstrates a one-shot plugin (fresh process per query).

- **Location:** `examples/plugins/case-converter-node/`
- **Keyword:** `case`
- **How to use:** Type `case hello` to see case-conversion options.

Enable and disable plugins

- **Enable:** Place the plugin folder in the plugins folder and reload.
- **Disable all script plugins:** In `config.toml`, add `"script"` to `[plugins] disabled`:

```
[plugins]
disabled = ["script"]
```

- **Disable a specific plugin:** Add its id (typically `script:<name>`) to the disabled list:

```
[plugins]
disabled = ["script:hello"]
```

Disabled plugins do not run, but their folders stay in place.

Script plugin types

Script plugins can work in two modes, defined in their `plugin.toml`:

Mode	How it works	When to use
Persistent (default)	One long-lived process speaks JSON lines on stdin/stdout	Complex plugins that benefit from caching or keeping state across queries
One-shot	A fresh process per query; it prints JSON and exits	Simple plugins, or quick ad-hoc scripts that do not need to stay running

A plugin's `plugin.toml` declares which mode it uses (`mode = "persistent"` or `mode = "oneshot"`). As a user, you do not need to know the difference—both work the same way.

Troubleshooting



Plugin did not load

1. Check that the plugins folder exists (create it if it does not).
2. Restart Sevak or choose **Reload index** from the tray.
3. Check Sevak's log for errors. Choose **Settings** → **Troubleshooting** (if available) or look at the log file (see [Files and data](#)).
4. Make sure the `plugin.toml` is valid TOML and has a `protocol`, `keyword`, and `script` or `command`.

Plugin asked for approval but I clicked the wrong thing

1. Note the plugin's id (shown in the approval prompt, e.g., `script:hello`).
2. Open `config.toml` and remove the id from `[plugins] disabled` if it was added.
3. Reload or restart Sevak.

Plugin appears but does not return results

1. Check the plugin's `plugin.toml` for the `script` or `command` field.
2. Verify the script file exists in the plugin folder.
3. On Linux and macOS, make sure the script is executable: `chmod +x script-file`.
4. Run the script manually from the plugin folder to check for errors (e.g., `python3 main.py hello` or `./main.py hello`).
5. Check Sevak's log for error messages.

Plugin hangs or is very slow

Script plugins have a timeout (soft: 50ms per query, hard: 3 seconds total). If your script is slow: 1. Check the plugin's `plugin.toml` for `timeout_ms` and `hard_timeout_ms` settings. 2. Optimize the script to respond faster (e.g., cache results, avoid network calls on every query). 3. For persistent plugins, make sure it is staying running between queries (check Sevak's log).

Share a script plugin

If you write a useful plugin, consider sharing it. Open an issue on the Sevak GitHub repository or link it in Discussions.

Learn more

For details on writing your own script plugin, see [Writing plugins](#).

Reference

Configuration file reference

Sevak stores settings in a TOML file. You can edit it by hand or use the Settings window. This page documents every configuration key.

Where the file is

Windows

```
%APPDATA%\sevak\config.toml
```

Example: `C:\Users\Ninad\AppData\Roaming\sevak\config.toml`

macOS

```
~/Library/Application Support/sevak/config.toml
```

Linux

```
~/.config/sevak/config.toml
```

You can override the config folder with the `SEVAK_CONFIG_DIR` environment variable or the `--config` flag. The Settings window's "Open config file" button opens it in your editor.

Editing and reloading

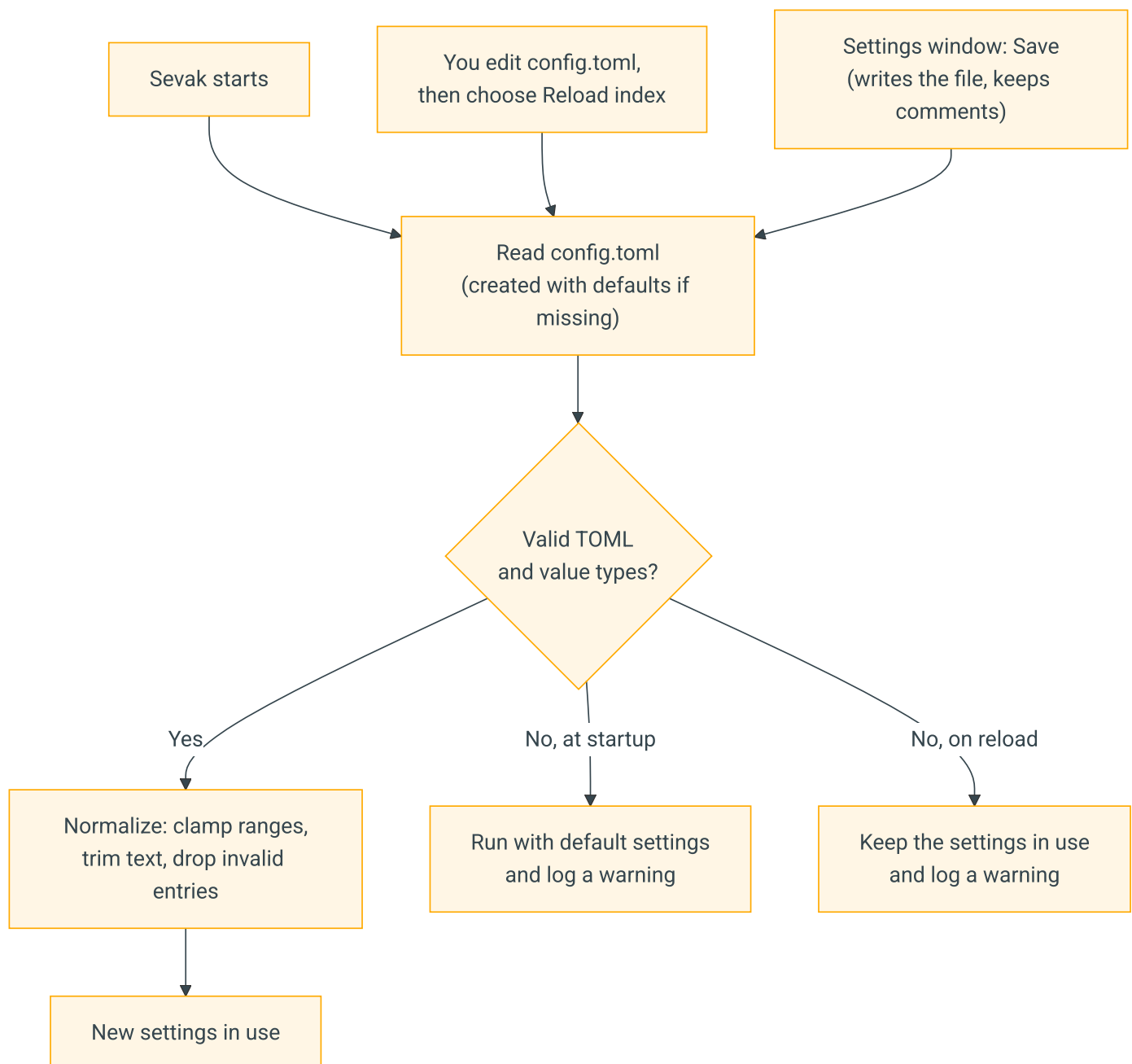
After making changes to `config.toml`:

- Choose **Reload index** from the tray menu or menu bar (fastest).
- Or restart Sevak.

If the file has a mistake (invalid TOML or a value of the wrong type), Sevak never overwrites it. At startup it runs with the default settings; on **Reload index** it keeps the settings already in use. Either way the problem is written to the [log](#), so check there if a change seems to have no effect. Comments and key order are preserved when Settings saves changes.

A minimal config works: missing fields use their defaults, so a nearly empty file is valid.

How configuration is loaded



Configuration sections

[general]

Main hotkeys and startup behaviour.

Key	Type	Default	Description
hotkey	string	"Alt+Space"	Global keyboard shortcut to show/hide Sevak. Examples: "Ctrl+Space", "Super+Space", "Ctrl+Shift+K". On

Key	Type	Default	Description
			macOS, <code>Alt</code> is the Option key. On Linux Wayland, run <code>sevak --setup-hotkey</code> to bind this in GNOME instead.
<code>actions_hotkey</code>	string	<code>"Ctrl+Alt+Space"</code>	Hotkey for Universal Actions: capture the selection in the foreground app and offer actions on it. Empty string <code>" "</code> turns Universal Actions off. On Wayland, run <code>sevak --setup-hotkey</code> to bind this in GNOME.
<code>hide_on_blur</code>	boolean	<code>true</code>	Hide the launcher when it loses focus to another window. Press Esc or click elsewhere to close; this setting hides it automatically.
<code>launch_at_login</code>	boolean	<code>false</code>	Start Sevak when you log in to your desktop.
<code>check_for_updates</code>	boolean	<code>true</code>	Check GitHub for a new version at startup and once daily. Updates are only installed after you confirm. Apart from optional currency rates, this is the only automatic network request.

```
[general]
hotkey = "Alt+Space"
actions_hotkey = "Ctrl+Alt+Space"
hide_on_blur = true
launch_at_login = false
check_for_updates = true
```



[window]

Launcher window appearance.

Key	Type	Default	Range	Description
<code>width</code>	integer	<code>720</code>	400–1600 (logical pixels)	Width of the search bar window. Logical pixels scale with the display's DPI.

`[window]
width = 720`

[linux]

Linux-specific settings (ignored on Windows and macOS).

Key	Type	Default	Description
<code>wayland_use_xwayland</code>	boolean	<code>true</code>	On Wayland sessions, draw Sevak through XWayland (X11 compatibility layer). Native Wayland windows cannot position themselves or request focus reliably, so XWayland keeps the launcher centred and typeable. Set to <code>false</code> to use the native Wayland backend (window may appear off-centre and not accept input). Only used on Wayland; X11 sessions ignore this.

`[linux]
wayland_use_xwayland = true`

[search]

Search behaviour and fallback when no results match.

Key	Type	Default	Range	Description
<code>max_results</code>	integer	<code>8</code>	1–20	Maximum number of results shown at once.

Key	Type	Default	Range	Description
<code>fallback_web_search</code>	string or list	<code>"g"</code>	any keyword from <code>[[web_search]]</code> entries	When a query has no local results, offer a web search with this engine. Write as a single string <code>"g"</code> or a list to try several in order: <code>["g", "yt", "gh"]</code> . Empty string <code>" "</code> disables the fallback. Keywords are matched case-insensitively; only one keyword per engine.
<code>query_history</code>	boolean	<code>true</code>	—	Remember the last 50 queries you ran. Press Up/Down on an empty search box to recall them. <code>false</code> disables history and clears it.

```
[search]
max_results = 8
fallback_web_search = "g"
query_history = true
```

[appearance]

Launcher theme and styling.

Key	Type	Default	Range/Values	Description
<code>theme</code>	string	<code>"system"</code> <code>" "</code>	<code>"system"</code> , <code>"light"</code> , <code>"dark"</code>	<code>"system"</code> follows your OS dark-mode preference. Invalid values fall back to <code>"system"</code> and are reported in the log and Settings.

Key	Type	Default	Range/Values	Description
<code>accent</code>	string	<code>" "</code> (empty)	<code>"#rgb"</code> , <code>"#rrggbb"</code> , <code>"rgb(r, g, b)"</code> , or <code>" "</code>	Override the theme's accent colour for buttons and highlights. <code>" "</code> uses the theme's own accent. Invalid values fall back to <code>" "</code> with a warning.
<code>font_size</code>	integer	<code>15</code>	12–22 (pixels)	Size of result titles. The rest of the launcher (icons, text, spacing) scales with it. Clamped to the range.
<code>font_family</code>	string	<code>" "</code> (empty)	any font family name, comma-separated	Comma-separated font family list, e.g. <code>"Fira Sans, sans-serif"</code> . Empty uses your system font. CSS font-family syntax; if the font is missing, the next in the list is used.
<code>opacity</code>	integer	<code>100</code>	30–100 (percent)	Background opacity of the search bar. 100 is fully opaque; 30 is quite transparent. Whole numbers only.
<code>radius</code>	integer	<code>14</code>	0–32 (pixels)	Corner radius of the search bar. 0 is sharp corners; 32 is very rounded. Whole numbers only.
<code>custom_css</code>	string	<code>" "</code> (empty)	filename in config folder, or <code>" "</code>	Stylesheet to override theme colours and layout. Must be a file in the config folder (same folder as <code>config.toml</code>), e.g. <code>"theme.css"</code> . <code>" "</code> loads none. See Themes guide for available CSS variables.

```
[appearance]
theme = "system"
accent = ""
font_size = 15
font_family = ""
opacity = 100
radius = 14
custom_css = ""
```



[plugins]

Which plugins are active. Each built-in plugin can be disabled.

Key	Type	Default	Description
<code>disabled</code>	array of strings	<code>[]</code>	Plugin ids to turn off. Available ids: <code>"apps"</code> , <code>"calculator"</code> , <code>"files"</code> , <code>"bookmarks"</code> , <code>"system"</code> , <code>"shell"</code> , <code>"clipboard"</code> , <code>"snippets"</code> , <code>"selection"</code> (Universal Actions), <code>"web:<keyword>"</code> (specific web-search engines). Example: <code>disabled = ["files", "web:yt"]</code> turns off file search and YouTube search. Unknown ids are ignored.

```
[plugins]
disabled = []
```



[calculator]

Calculator and unit conversion settings.

Key	Type	Default	Description
<code>currency</code>	boolean	<code>false</code>	Enable currency conversion (e.g. <code>100 usd in eur</code>). Off by default because it requires network access: Sevak downloads the European Central Bank's daily reference rates (a small XML file, no API key) at most once per day and caches them. Unit conversion (e.g. <code>10 km in mi</code>) never needs the network and is always on.

```
[calculator]
currency = false
```



[files]

Indexed file search settings.

Key	Type	Default	Description
<code>directories</code>	array of strings	<code>["~/Desktop", "~/Documents", "~/Downloads"]</code>	Root folders to index recursively. <code>~</code> expands to your home directory and works on all platforms. Relative paths are anchored at the working directory when Sevak starts (usually your home). Forward slashes work everywhere; avoid unescaped backslashes. The index caps at 100,000 entries; reduce roots or depth if you hit this.
<code>max_depth</code>	integer	<code>4</code>	How many folder levels below each root are indexed. <code>1</code> means only files directly in the root.
<code>include_hidden</code>	boolean	<code>false</code>	Index dot-files and dot-folders (starting with <code>.</code>). <code>false</code> ignores them.
<code>keyword</code>	string	<code>"f"</code>	Prefix to search only files: type <code>f filename</code> . Leave empty <code>" "</code> to disable keyword search.
<code>global</code>	boolean	<code>true</code>	Show file results even in ordinary queries without the keyword. <code>false</code> requires <code>f <name></code> to search files.

```
[files]
directories = [ "~/Desktop", "~/Documents", "~/Downloads" ]
max_depth = 4
include_hidden = false
keyword = "f"
global = true
```



[bookmarks]

Browser bookmark search settings.

Key	Type	Default	Description
browsers	array of strings	[]	Browser profiles to index. Empty [] means every browser found on your system. Supported ids: "chrome", "edge", "brave", "vivaldi", "chromium", "opera", "opera-gx", "firefox", "librewolf", "zen". All profiles of each browser are searched. Unknown ids are ignored.
keyword	string	"b"	Prefix to search only bookmarks: type b term. Leave empty "" to disable keyword search.
global	boolean	true	Show bookmark results in ordinary queries without the keyword. false requires b <term> to search bookmarks.

```
[bookmarks]
browsers = []
keyword = "b"
global = true
```



[system]

System commands (lock, sleep, restart, etc.) and settings pages.

Key	Type	Default	Description
confirm	boolean	true	Ask for confirmation before restart, shut down, log out and emptying the trash. Power commands are destructive so default to "ask".
disabled	array of strings	[]	Commands and settings pages to hide. Options: "lock", "sleep", "hibernate", "restart", "shutdown", "logout", "empty_trash", "settings" (all settings pages), or specific pages like "settings:bluetooth", "settings:wifi". Case-insensitive. Unknown entries are ignored. To turn off the entire plugin, add "system" to [plugins] disabled instead.

```
[system]
confirm = true
disabled = []
```



[shell]

Shell command execution settings (type `> <command>` to run a command).

Key	Type	Default	Description
<code>terminal</code>	string	<code>""</code> (auto-detect)	Terminal program to run commands in. Empty means auto-detect: Windows Terminal (else a console window) on Windows, Terminal.app on macOS, <code>\$TERMINAL</code> environment variable then common terminals on Linux. You can give a program name (<code>"wt"</code> , <code>"iterm"</code> , <code>"kitty"</code>), a full path, or a command with arguments (<code>"alacritty --class sevak"</code>).
<code>shell</code>	string	<code>""</code> (auto-detect)	Shell that runs the command. Empty means auto-detect: <code>pwsh</code> , then <code>powershell</code> , then <code>cmd</code> on Windows; <code>\$SHELL</code> environment variable or <code>sh</code> on Linux. Ignored on macOS, where the terminal starts your login shell. You can give a program name or full path.
<code>keep_open</code>	boolean	<code>true</code>	Keep the terminal open at a shell prompt after the command exits. <code>false</code> closes it when done.

```
[shell]
terminal = ""
shell = ""
keep_open = true
```



[paste]

How text is pasted into the previously focused app.

Key	Type	Default	Description
<code>restore_clipboard</code>	boolean	<code>false</code>	After pasting from clipboard history or snippets, restore the previous clipboard contents. <code>false</code> leaves the pasted text in the clipboard.

```
[paste]
restore_clipboard = false
```



[actions]

Universal Actions: how the selection in another app is captured.

Key	Type	Default	Description
<code>use_primary_selection</code>	boolean	<code>true</code>	Linux X11 only: read the PRIMARY selection (highlighted text) first, without pressing Ctrl+C. Ignored on Wayland (no PRIMARY selection) and other platforms.
<code>use_clipboard_fallback</code>	boolean	<code>false</code>	When the selection cannot be captured (Wayland, terminal windows, missing macOS Accessibility permission), act on the current clipboard contents instead. <code>true</code> adds a fallback; <code>false</code> does nothing if capture fails.

```
[actions]
use_primary_selection = true
use_clipboard_fallback = false
```



[clipboard]

Clipboard history settings (opt-in feature).

Key	Type	Default	Description
<code>enabled</code>	boolean	<code>false</code>	Enable clipboard history (<code>cb <text></code> to search). Off by default: turning it on makes Sevak watch your clipboard and keep copied text in a local file. Text only; content marked as secret by apps (password managers) is never recorded.
<code>max_items</code>	integer	<code>200</code>	How many clipboard items to keep. Older items are dropped. Range: 1–5000.
<code>max_item_bytes</code>	integer	<code>65536</code> (64 KiB)	Maximum size of a clipboard item in bytes. Longer text is not recorded. Range: 1–4,194,304 (4 MiB).
<code>ignore_apps</code>	array of strings	<code>[]</code>	Apps whose copies are never recorded, e.g. <code>["KeePassXC", "1Password"]</code> . Matched case-insensitively against the program or app name.

```
[clipboard]
enabled = false
max_items = 200
max_item_bytes = 65536
ignore_apps = []
```



[[snippet]]

Snippets: text you paste often, with placeholders that expand.

Key	Type	Required	Description
<code>name</code>	string	yes	Name of the snippet; type <code>s <name></code> to search for it. Must not be empty.
<code>text</code>	string	yes	Text to paste. Can include newlines (<code>\n</code>). Must not be empty.

Key	Type	Required	Description
<code>keyword</code>	string	optional	Extra word to search by, e.g. <code>keyword = "sig"</code> lets you find it with <code>s sig</code> . Trimmed; empty keywords are ignored.

Placeholders in `text`:

- `{date}` — today's date in YYYY-MM-DD format
- `{time}` — current time in HH:MM:SS format (24-hour)
- `{datetime}` — date and time, e.g. 2025-10-03 14:30:00
- `{date:%d %B %Y}` — date in a custom format, e.g. 03 October 2025 (uses [Rust strftime syntax](#))
- `{clipboard}` — contents of the clipboard at paste time
- `{uuid}` — a random UUID v4
- `{{ and }}` — literal `{` and `}` in the output

The Settings window never reformats hand-written snippets, so they survive a save. Unknown placeholders are left as-is.

```
[[snippet]]
name = "Email signature"
keyword = "sig"
text = "Best regards, \nNinad\n{date}"

[[snippet]]
name = "Meeting note"
text = "{datetime}: {clipboard}"
```

[[web_search]]

Web search engines. Type the keyword followed by your search terms, e.g. `g rust traits`.

Key	Type	Required	Description
<code>keyword</code>	string	yes	Short name to type before the query, e.g. <code>"g"</code> for Google. Must be unique and contain no spaces. Trimmed.
<code>name</code>	string	yes	Display name shown in results, e.g. <code>"Google"</code> .

Key	Type	Required	Description
<code>url</code>	string	yes	URL template. Must contain <code>{query}</code> (replaced by the URL-encoded search terms) and start with <code>http://</code> or <code>https://</code> .

If you define any `[[web_search]]` entries, they replace the defaults entirely. The defaults are:

```
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"
```

[[hotkey]]

Extra global hotkeys: bind a key to open Sevak with a query or run a result directly.

Key	Type	Required	Description
<code>key</code>	string	yes	Keyboard shortcut, e.g. <code>"Ctrl+Alt+T"</code> . Parsed like <code>[general] hotkey</code> . Must not be empty.
<code>query</code>	string	optional*	Open Sevak with this text already typed. Example: <code>query = ">"</code> opens a shell command prompt. Either <code>query</code> or <code>run</code> , not both.
<code>run</code>	string	optional*	Run the result with this id without showing the launcher. Example: <code>run = "apps:firefox.desktop"</code> . Either <code>query</code> or <code>run</code> , not both. Must not be empty.

*Exactly one of `query` or `run` must be present.

Result ids:

- Apps: `apps:firefox.desktop` (Linux desktop-file id), `apps:Chrome` or `apps:AppUserModelID` (Windows), `apps:Safari` (macOS)
- Files: `files:<full path>`
- Bookmarks: hover over a bookmark in Sevak to see its id
- System commands: `system:lock`, `system:sleep`, `system:restart`, `system:shutdown`, `system:logout`, `system:settings`, `system:settings:bluetooth`, `system:settings:wifi`, etc.
- Snippets: `snippets:<name>`
- Shell commands: `shell:<command>`

Calculator, web search and clipboard history results have no stable id and cannot be bound.

On Wayland, run `sevak --setup-hotkey` after editing to create GNOME desktop shortcuts. Delete old entries in GNOME's keyboard settings after removing them from `config.toml`. On X11 and other desktops, the keys are registered directly by Sevak.

```
[[hotkey]]
key = "Ctrl+Alt+T"
query = "> "

[[hotkey]]
key = "Ctrl+Alt+L"
run = "system:lock"

[[hotkey]]
key = "Super+F"
run = "apps:firefox.desktop"
```

Complete example

This is the full default configuration written on first run:



```
# Sevak configuration
#
# Created with default values on first run. Edit it, then choose "Reload index"
# from the tray menu (or restart Sevak) to apply changes.

[general]
# Shortcut that shows and hides Sevak. Examples: "Alt+Space", "Ctrl+Space",
# "Super+Space", "Ctrl+Shift+K".
# On Linux Wayland sessions applications cannot grab global keys. Run
# `sevak --setup-hotkey` to bind this key to `sevak --toggle` in GNOME instead.
hotkey = "Alt+Space"

# Shortcut for Universal Actions: it copies what you have selected in the app
# you are using (text, a URL, files) and offers actions for it. "" turns it off.
# On Wayland run `sevak --setup-hotkey` to bind it to `sevak --actions`. See [actions].
actions_hotkey = "Ctrl+Alt+Space"

# Hide the window when it loses focus.
hide_on_blur = true

# Start Sevak in the background when you log in.
launch_at_login = false

# Check GitHub for a new version at startup and once a day. Updates are only
# installed after you agree. Apart from the optional currency rates (see
# [calculator]), this is the only request Sevak makes on its own.
check_for_updates = true

[window]
# Width of the search window in logical pixels (400-1600).
width = 720

[linux]
# On Wayland sessions, draw Sevak's window through XWayland. Native Wayland
# windows cannot position themselves and may be refused focus, so this keeps the
# launcher centered and typeable. Set to false to use the native Wayland backend.
wayland_use_xwayland = true

[search]
# Number of results shown (1-20).
max_results = 8
# Keyword of the web search engine offered when nothing else matches (" to
# disable). A list offers several, in order: ["g", "yt", "gh"].
fallback_web_search = "g"
# Up/Down on an empty search box recalls the last 50 searches you ran. They are
# kept in usage.json in the data folder; false stops recording and forgets them.
query_history = true

[appearance]
# "system", "light" or "dark".
theme = "system"
# Accent color as "#rrggbb", "#rgb" or "rgb(r, g, b)". "" keeps the theme's own.
accent = ""
# Size of the result titles in pixels (12-22); the rest of the bar scales with it.
font_size = 15
# Font for the search bar, e.g. "Fira Sans, sans-serif". "" uses the system font.
font_family = ""
# How opaque the search bar's background is, in percent (30-100).
```

```

opacity = 100
# Corner radius of the search bar in pixels (0-32).
radius = 14
# A stylesheet inside this config folder that overrides the theme's CSS variables
# (see docs/themes.md), for example "theme.css". "" loads none.
custom_css = ""

[plugins]
# Ids of built-in plugins to turn off: "apps", "calculator", "files",
# "bookmarks", "system", "shell", "clipboard", "snippets", "selection"
# (Universal Actions), "web:<keyword>".
disabled = []

[calculator]
# Convert currencies ("100 usd in eur"). Off by default because it needs the
# network: when on, Sevak downloads the European Central Bank's daily reference
# rates (a small XML file, no account or key) in the background at most once a
# day and keeps them on disk. Unit conversion ("10 km in mi") never needs it.
currency = false

[files]
# Folders whose files and subfolders are searchable. "~" is your home folder.
directories = ["~/Desktop", "~/Documents", "~/Downloads"]
# How many folder levels below each directory are indexed.
max_depth = 4
# Index dot-files and dot-folders.
include_hidden = false
# Type "<keyword> <name>" to search only files.
keyword = "f"
# Also show (lower-ranked) file results for plain queries.
global = true

[bookmarks]
# Browsers whose bookmarks are searchable; [] means every browser found.
# Names: "chrome", "edge", "brave", "vivaldi", "chromium", "opera", "opera-gx",
# "firefox", "librewolf", "zen". All profiles of each browser are read.
browsers = []
# Type "<keyword> <text>" to search only bookmarks.
keyword = "b"
# Also show (lower-ranked) bookmark results for plain queries.
global = true

[system]
# System commands: lock, sleep, hibernate, restart, shut down, log out, empty
# the trash, and shortcuts to OS settings pages. Ask before restart, shut down,
# log out and emptying the trash.
confirm = true
# Commands or pages to hide: "lock", "sleep", "hibernate", "restart",
# "shutdown", "logout", "empty_trash", "settings" (every settings page) or
# "settings:<page>" such as "settings:bluetooth". To turn the whole plugin off,
# add "system" to [plugins] disabled instead.
disabled = []

[shell]
# Type "> <command>" (or ">command") to run a command in a terminal window. It
# only runs when you press Enter; recent commands are offered again.
# Terminal to use. "" detects one: Windows Terminal (else a console window) on
# Windows, Terminal.app on macOS, $TERMINAL then common terminals on Linux.
# Examples: "wt", "iterm", "kitty", "gnome-terminal", "alacritty --class sevak".

```

```

terminal = ""
# Shell that runs the command. "" picks pwsh, powershell, then cmd on Windows
# and $SHELL (or sh) on Linux. Not used on macOS: your login shell runs it.
shell = ""
# Leave the terminal open, at a shell prompt, after the command exits.
keep_open = true

[paste]
# Clipboard history and snippets paste into the app you were using before Sevak
# opened. With this on, the clipboard's previous text is put back afterwards.
restore_clipboard = false

[actions]
# Universal Actions (see actions_hotkey under [general]). Sevak presses Ctrl+C
# (Cmd+C) in the app you were using, reads the result and puts your clipboard
# back. The selection is never stored or logged. Terminal windows are skipped
# on Windows and Linux because Ctrl+C would interrupt the running program.
# Linux (X11): read the PRIMARY selection (text you just highlighted) first,
# without pressing a key.
use_primary_selection = true
# When the selection cannot be captured (Wayland, a terminal, missing macOS
# Accessibility permission), act on the current clipboard contents instead.
use_clipboard_fallback = false

[clipboard]
# Clipboard history ("cb <text>"). Off by default: turning it on makes Sevak
# watch the clipboard and keep copied text in clipboard-history.json in its data
# folder. Text only. Content that apps mark as secret (password managers) is
# never recorded.
enabled = false
# Items kept (the oldest are dropped).
max_items = 200
# Longer text is not recorded.
max_item_bytes = 65536
# Never record text copied from these apps, e.g. ["KeePassXC", "1Password"].
# Matched case-insensitively against the program or app name.
ignore_apps = []

# Snippets ("s <name>"): text you paste often. Placeholders: {date}, {time},
# {datetime}, {date:%d %B %Y}, {clipboard}, {uuid}; write {{ and }} for literal
# braces. "keyword" is optional and also matches the search.
# [[snippet]]
# name = "Email signature"
# keyword = "sig"
# text = "Best regards,\nNinad"

# Web search engines: type "<keyword> <terms>". "{query}" is replaced by the
# URL-encoded terms. Defining any [[web_search]] entry replaces this list.
[[web_search]]
keyword = "g"
name = "Google"
url = "https://www.google.com/search?q={query}"

[[web_search]]
keyword = "yt"
name = "YouTube"
url = "https://www.youtube.com/results?search_query={query}"

[[web_search]]

```

```
keyword = "gh"
name = "GitHub"
url = "https://github.com/search?q={query}"

# Extra global hotkeys. Each [[hotkey]] has a "key" and exactly one of:
#   query = "..." open Sevak with this text already typed
#   run = "..." run a result directly, without showing Sevak; the value is a
#               result id such as "apps:firefox.desktop" or "files:<full path>"
# On Linux Wayland, `sevak --setup-hotkey` binds these in GNOME as well.
#
# [[hotkey]]
# key = "Ctrl+Alt+T"
# query = "> "
#
# [[hotkey]]
# key = "Ctrl+Alt+F"
# run = "apps:firefox.desktop"
```

Command-line reference

Sevak can be controlled from the command line or automated with scripts. When Sevak is already running, a new command forwards its request to the running instance (single-instance architecture).

Quick reference

Flag	Effect
(none)	Start Sevak; show the search bar if not already visible
<code>--toggle</code>	Show the search bar, or hide it if visible
<code>--query TEXT</code>	Show the search bar with TEXT already typed
<code>--run ID</code>	Run a result by its id without showing the search bar
<code>--actions</code>	Universal Actions: capture the selection in the foreground app
<code>--background</code>	Start Sevak without showing the window
<code>--settings</code>	Open the Settings window
<code>--quit</code>	Quit the running instance
<code>--setup-hotkey [KEY]</code>	Bind hotkeys to GNOME (Wayland desktop on Linux)
<code>--config PATH</code>	Use a custom config folder or file; overrides <code>SEVAK_CONFIG_DIR</code>
<code>-h, --help</code>	Print help
<code>-V, --version</code>	Print version

Environment variables

Variable	Purpose
SEVAK_CONFIG_DIR	Path to the config folder (overridden by <code>--config</code>). If this is set and the folder exists, the config is read from there.
SEVAK_DATA_DIR	Path to the data folder (usage statistics, logs). If set, overrides the default platform-specific location.
SEVAK_LOG	Log level filter. Examples: <code>SEVAK_LOG=debug</code> for detailed output, <code>SEVAK_LOG=warn</code> for warnings and errors only. Defaults to <code>info</code> .

Flags

Start or show

(no arguments)

```
sevak
```

Start Sevak in the background if it is not running. If it is already running and the search bar is hidden, show it. Does nothing if the bar is already visible.

`--toggle`

```
sevak --toggle
```

Toggle the search bar: show it if hidden, hide it if visible. This is the primary way to bind Sevak to a global hotkey.

On Wayland, where applications cannot grab global keys, bind this to a key in your desktop settings (or use `sevak --setup-hotkey` on GNOME).

`--query TEXT`

```
sevak --query "firefox"
sevak --query "> pip install requests"
sevak --query ""
```

Show the search bar with TEXT already typed. Used to:

- Open Sevak with a specific search, e.g. `"firefox"` to search for Firefox
- Pre-fill a shell prompt with `>` or `>` (see [Shell commands](#))
- Open an empty launcher with an empty string `""`

--background

```
sevak --background
```

Start Sevak in the background. The search bar does not appear. Useful for the `launch_at_login` setting or autostart scripts.

--run ID

```
sevak --run "apps:firefox.desktop"  
sevak --run "files:/home/user/Documents"  
sevak --run "system:lock"
```

Run a result by its id directly without showing the search bar. The launcher stays hidden. Useful for hotkey bindings, e.g. bind `Ctrl+Alt+L` to `sevak --run "system:lock"`.

See [custom hotkeys](#) for result id formats.

--actions

```
sevak --actions
```

Universal Actions: capture what you have selected in the currently focused app (text, a URL, files) and offer actions on it. On Wayland, bind this to a key in your desktop settings or use `sevak --setup-hotkey` on GNOME.

Interaction

--settings

```
sevak --settings
```

Open the Settings window. If Sevak is not running, start it with the Settings window shown.

--quit

```
sevak --quit
```

Quit the running instance. If Sevak is not running, this does nothing and exits cleanly.

Configuration

--config PATH

```
sevak --config ~/.local/share/sevak  
sevak --config /etc/sevak/config.toml  
sevak --config .\Config
```

Use a custom config folder or config file instead of the default location. Takes precedence over the `SEVAK_CONFIG_DIR` environment variable.

- If `PATH` ends in `.toml` and is not an existing directory, it is treated as the config file itself; the folder containing it is the config folder.
- Otherwise, `PATH` is treated as the config folder; the file is `config.toml` inside it.
- `~` is expanded to your home directory.
- Relative paths are resolved relative to the current working directory.

This flag only affects a newly started instance. If Sevak is already running, it keeps its own config folder; the flag is noted in the log for reference.

`--setup-hotkey [KEY]`

(Linux Wayland with GNOME only)

```
sevak --setup-hotkey          # use the hotkey from config.toml
sevak --setup-hotkey "Ctrl+Space"
sevak --setup-hotkey "Super+F"
```

Create GNOME custom keyboard shortcuts that bind your hotkeys to Sevak commands. On Wayland, where applications cannot register global keys, this command asks GNOME to handle key events and forward them to Sevak.

It creates shortcuts for:

- The main hotkey (`[general] hotkey`) → `sevak --toggle`
- Universal Actions hotkey (`[general] actions_hotkey`) → `sevak --actions`
- Each `[[hotkey]]` entry → `sevak --query '<text>'` or `sevak --run <id>`

If you pass a `KEY`, it overrides the hotkey from `config.toml` just for this setup. The config file is not modified.

On other desktops (KDE, Sway, etc.), bind hotkeys through your desktop's keyboard settings. On X11, hotkeys are registered directly by Sevak and this command is not needed.

After editing `config.toml`, run `--setup-hotkey` again to update GNOME shortcuts. Delete old shortcuts in GNOME's keyboard settings if you remove entries from the config.

Information

`-h, --help`

```
sevak --help
sevak -h
```

Print the help message and exit.

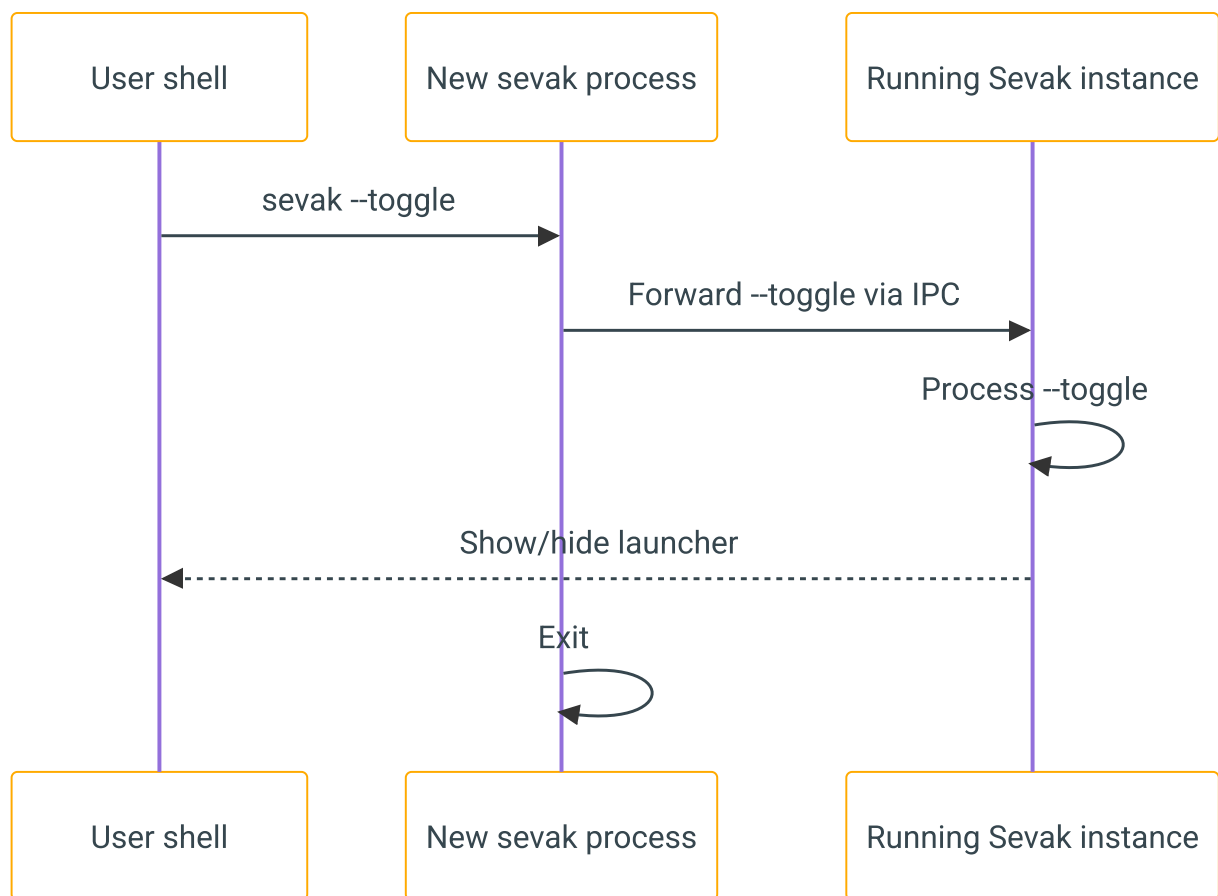
`-V, --version`

```
sevak --version
sevak -V
```

Print the version and exit.

Single-instance forwarding

When you run `sevak` and the app is already running, your command is forwarded to the running instance. Here is how it works:



This means:

- Only one instance of Sevak runs at a time.
- Commands from new invocations are processed by the running instance.
- If Sevak is not running, the new invocation starts it.
- The `--config` flag is ignored by the running instance (it keeps its own config path).

Exit codes:

Code	Meaning
0	Success (including <code>--help</code> and <code>--version</code>)
1	Sevak could not start or run (the error is printed and logged)
2	Invalid command line: unknown flag, missing value or conflicting flags (the usage text is printed)

Examples

Bind Alt+Space to show/hide Sevak on macOS:

Add to your keyboard shortcuts under System Settings:

```
/Applications/Sevak.app/Contents/MacOS/sevak --toggle
```



Bind Ctrl+Alt+T to open a shell prompt on Windows:

Use Task Scheduler or a tool like AutoHotkey to run:

```
sevak --query "> "
```



Lock the desktop from anywhere:

```
sevak --run "system:lock"
```



Search for a specific file from a script:

```
sevak --query "f my-project"
```



Start Sevak on login (Linux):

Add to your `.bash_profile` or autostart script:

```
sevak --background
```



Check Sevak version in a script:

```
sevak --version
```



Development and debugging

Enable debug logging:

Bash/Linux/macOS

```
SEVAK_LOG=debug sevak
```



PowerShell/Windows

```
$env:SEVAK_LOG = "debug"  
sevak
```



Windows CMD

```
set SEVAK_LOG=debug  
sevak
```



Logs are written to:

Windows

```
%APPDATA%\sevak\logs\
```

macOS

```
~/Library/Application Support/sevak/logs/
```

Linux

```
~/local/share/sevak/logs/
```

Test a config file before deployment:

```
sevak --config /path/to/test/config.toml --version
```



This loads the config without starting the UI. Parsing errors are printed to the terminal.

Files and data locations

Sevak stores configuration, logs, usage history and clipboard history in well-known locations on each platform. You can override these with environment variables or command-line flags.

Default locations

Item	Windows	macOS	Linux
Config folder	%APPDATA%\sevak\	~/Library/Application Support/sevak/	~/.config/sevak/
Config file	%APPDATA%\sevak\config.toml	~/Library/Application Support/sevak/config.toml	~/.config/sevak/config.toml
Data folder	%APPDATA%\sevak\	~/Library/Application Support/sevak/	~/.local/share/sevak/
Logs	%APPDATA%\sevak\logs\	~/Library/Application Support/sevak/logs/	~/.local/share/sevak/logs/
Usage history	%APPDATA%\sevak\usage.json	~/Library/Application Support/sevak/usage.json	~/.local/share/sevak/usage.json
Clipboard history	%APPDATA%\sevak\clipboard-history.json	~/Library/Application Support/sevak/clipboard-history.json	~/.local/share/sevak/clipboard-history.json

Expand ~ to your home directory and %APPDATA% to your roaming app data folder.

Override locations

Config folder

Command line

```
sevak --config ~/my-sevak  
sevak --config /etc/sevak/config.toml
```

Windows

```
sevak --config %env:USERPROFILE%\Sync\sevak
```

Environment variable

```
export SEVAK_CONFIG_DIR=~/my-sevak  
sevak
```

Windows

```
%env:SEVAK_CONFIG_DIR = "%env:USERPROFILE%\Sync\sevak"  
sevak
```

The `--config` flag takes precedence over the environment variable.

Data folder (logs, usage, clipboard history)

Environment variable

```
export SEVAK_DATA_DIR=~/.local/share/my-sevak  
sevak
```

Windows

```
%env:SEVAK_DATA_DIR = "D:\Data\sevak"  
sevak
```

Note: the data folder cannot be overridden via command line; only the `SEVAK_DATA_DIR` environment variable works.

What each file contains

config.toml

Your configuration: shortcuts, appearance, enabled plugins, file search roots, etc. See [Configuration file reference](#).

Edited by: You (manually or through the Settings window).

When to back up: Before making large manual edits or testing unusual configurations.

logs/

Day-to-day logs from Sevak (one file per day, dated). Useful for troubleshooting. For more detail, set `SEVAK_LOG=debug` before starting Sevak.

Automatic cleanup: Logs older than 7 days are deleted automatically.

Size: Typically a few kilobytes per day under normal use.

usage.json

Records searches and actions you perform: what you typed, which results you ran, timestamps. Used for:

- Query history: press Up/Down on an empty search bar to recall your last ~50 searches.
- Usage statistics and analytics (local only; nothing is sent to Sevak developers).

Edited by: Sevak (you can manually edit it, but Sevak will overwrite it).

Privacy: Text you searched is recorded. Select what to delete (see "Clear history" below).

clipboard-history.json

Optional clipboard history (only if `[clipboard] enabled = true`). Records text you copy from most apps, with timestamps.

Edited by: Sevak (when you copy text) and the Settings window (when you delete items).

Privacy: Disabled by default because it records what you copy. Apps that mark their content as secret (password managers, etc.) are never recorded. Configure `[clipboard] ignore_apps` to never record copies from specific apps.

Back up your data

Sevak's configuration and history are local, single files that are easy to back up:

```
cp -r ~/.local/share/sevak ~/backups/sevak-backup
```

or on Windows:

```
Copy-Item $env:APPDATA\sevak $env:USERPROFILE\backups\sevak-backup -Recurse
```

Restore from a backup by copying the folder back.

Reset to defaults

To reset Sevak to a fresh install while keeping your data:

1. Delete `config.toml` (or move it aside first). Sevak will write a new default config on next start.

2. Start Sevak.

To also clear history:

1. Delete `config.toml`.
2. Delete `usage.json` (or just the queries and actions you want to forget).
3. If clipboard history is enabled, delete `clipboard-history.json`.
4. Start Sevak.

Clear search history

Search history is stored in `usage.json`. To clear it entirely:

```
rm ~/.local/share/sevak/usage.json
```

On Windows:

```
Remove-Item "$env:APPDATA\sevak\usage.json"
```

Sevak will create a fresh file on next start.

To selectively delete searches, edit `usage.json` in a text editor (it is JSON format) and remove entries from the `searches` array.

Clear clipboard history

To delete all saved clipboard items:

1. Open Sevak Settings.
2. Go to Clipboard.
3. Use the "Delete all" button, or manually delete the file:

```
rm ~/.local/share/sevak/clipboard-history.json
```

On Windows:

```
Remove-Item "$env:APPDATA\sevak\clipboard-history.json"
```

Sevak will create a fresh file on next use.

Uninstall cleanly

Windows

1. Open Settings → Apps → Installed apps, find Sevak, and click Uninstall.
2. The installer removes Sevak itself but keeps config and data.
3. To also remove config and data:

```
Remove-Item "$env:APPDATA\sevak" -Recurse
```



macOS

1. Delete the Sevak app from Applications.
2. This does not remove config and data.
3. To also remove config and data:

```
rm -rf ~/Library/Application\ Support/sevak
```



Linux

Uninstall using your package manager:

Ubuntu/Debian

```
sudo apt remove sevak
```



Fedora

```
sudo dnf remove Sevak
```



Other distributions

```
# Delete the AppImage  
rm ~/Sevak_*.AppImage
```



To also remove config and data:

```
rm -rf ~/.config/sevak ~/.local/share/sevak
```



Portable setup

You can place Sevak's config and data anywhere and point to it with environment variables:

```
export SEVAK_CONFIG_DIR=/mnt/portable/sevak/config  
export SEVAK_DATA_DIR=/mnt/portable/sevak/data  
sevak
```



or

```
sevak --config /mnt/portable/sevak/config
```



This is useful for running Sevak from a USB drive or shared network folder. The `--config` flag overrides the default but not the environment variable if both are set; the flag takes precedence.

Troubleshooting file access

"Cannot access config file" error:

- Check file permissions. Your user must be able to read and write the config folder.
- Check disk space. If the drive is full, writing a new config may fail.
- Check paths with special characters (non-ASCII, spaces). Ensure the path is valid on your system.

Logs are not being written:

- Check that `~/.local/share/sevak/logs/` (or equivalent) exists and is writable.
- Set `SEVAK_LOG=debug` to enable debug logging, but this is stored in memory unless persisted to a file.
- Restart Sevak after changing the data folder.

Old clipboard history or usage data still showing:

- Sevak loads these files at startup. Changes made externally are not live-reloaded.
- Restart Sevak after editing or deleting `usage.json` or `clipboard-history.json`.

Help

Troubleshooting

Start here

Before diving into specific issues, try these first:

1. **Confirm Sevak is running:** Look for the tray icon (Windows/Linux) or menu bar icon (macOS). If it is missing, open Sevak from your application menu or run `sevak` in Terminal.
2. **Check your hotkey:** Open Settings and look at the configured shortcut under General. Try a different key if the current one is already in use by another app or your desktop.
3. **Reload the index:** After installing a new app or changing file search settings, open the tray menu and choose **Reload index**.
4. **Check the logs:** If the problem persists, open Settings → "Reveal logs folder" and look at the most recent log file. Enable debug logging with `SEVAK_LOG=debug sevak` for more detail.

The hotkey does not work

Diagnosis: Open the tray/menu bar icon and click "Show". If the launcher appears, Sevak is running and the hotkey is not registered correctly.

Try these steps:

1. **Choose a different key** in Settings → General. Your current key may be in use by another app or your desktop.
2. **macOS:** The default is `⌘ Option + Space` (Alt and Option are the same key on Mac). If it doesn't work, Option+Space might be reserved by Spotlight or another app—try a different key.
3. **Linux Wayland (GNOME):** Sevak cannot register global keys on Wayland. Run this command:

```
sevak --setup-hotkey
```

This creates a GNOME custom keyboard shortcut. If you get a conflict warning, follow the instructions to free `⌘ Alt + Space` or choose another key.

1. **Linux Wayland (other desktops):** Bind `⌘ Alt + Space` or another key to `sevak --toggle` in your desktop's keyboard settings (KDE Settings, Sway config, etc.).
2. **Linux X11 on GNOME:** `⌘ Alt + Space` is reserved for the window menu. Try `^ Ctrl + Space` or `⌘ Super + Space` (but `⌘ Super + Space` switches input sources if you have multiple layouts). Or free `⌘ Alt + Space`:

```
gsettings set org.gnome.desktop.wm.keybindings activate-window-menu "[ ]"
```

There is no tray icon on Linux

Sevak shows a tray icon through AppIndicator. Some GNOME sessions do not have the AppIndicator extension enabled.

Ubuntu: The extension is enabled by default. If there is no tray icon, check that GNOME Extensions are working.

Fedora: Install and enable the AppIndicator extension:

```
sudo dnf install gnome-shell-extension-appindicator  
killall -HUP gnome-shell # or restart GNOME
```



Without a tray: Sevak still works. Use your hotkey to show it, `sevak --quit` to exit, and `sevak --settings` to open Settings.

An installed app is missing

- Choose **Reload index**, wait for indexing, and search again.
- Confirm the **Applications** plugin is enabled.
- Try the application's full name.
- Confirm the app has a platform launcher entry: a Start Menu shortcut or packaged app on Windows, an application bundle on macOS, or a desktop entry on Linux. Merely copying an executable into a folder may not create one.

A file or folder is missing

Check these in order:

Check	What to do
Query length	Use at least two characters after <code>f</code>
Indexed root	Add the containing folder in Settings → Files
Search depth	Increase depth if the file is nested too deeply
Dot-file visibility	Enable hidden-file indexing if needed
Excluded directory	Generated/cache directories are deliberately pruned

Check	What to do
New or moved file	Reload the index
Permissions	Ensure your user account can read that directory
Index size	Reduce broad roots if you hit the 100,000-entry cap

File search matches names, not document contents. A word appearing only inside a PDF or document is not a filename match. [Full file-search behavior](#).

I get a web suggestion instead of a local result

The fallback web engine is offered when an ordinary query has no matches. It does not mean your local query has already been sent to that provider. A browser request happens when you execute the web result.

Check local plugins and indexing first, or disable the fallback under **Settings** → **Search**.

A web keyword does not work

Use `g rust traits`, with a space after `g`. Then check that:

- The engine exists and is enabled.
- Its keyword is unique and has no spaces.
- Its URL starts with HTTP(S) and contains `{query}`.
- Your custom TOML engine list includes all the default engines you still want.

Defining `[[web_search]]` entries replaces the default list. See [Web search configuration](#) for examples and details.

A calculation looks unexpected

Check parentheses and operators. `%` means remainder; use `200*15/100` for 15% of 200. Powers have precedence over a leading minus, so `-2^2` is `-4`; use `(-2)^2` for `4`. Press Enter to **copy** the answer, then paste it where you need it.

Sevak uses finite-precision arithmetic. Invalid expressions and operations such as division by zero do not produce a valid copyable answer.

The launcher disappears

This is expected with **Hide on blur** enabled: the launcher hides when another window takes focus. Turn it off under Settings → General if you prefer. Pressing Esc also hides the launcher; **Quit** in the tray stops the app.

Settings will not save

The Settings window shows validation errors if there are problems. Fix them in order:

Common issues:

- Duplicate web search keywords (each must be unique)
- URL without `{query}` placeholder
- Invalid or empty hotkey
- Fallback web search engine that no longer exists
- Blank fields where text is required

Hand-edited TOML files:

Check for syntax errors (unmatched quotes, missing brackets, typos in section names). If the file can't be read, Sevak starts with default settings (or, on **Reload index**, keeps the settings already in use) and writes the reason to the log. See [Files and data](#) for the config path and how to back it up.

An update check fails

Confirm internet access and that GitHub Releases is reachable. The release must include a valid update manifest and a compatible package. Automatic checks run in release builds; development/debug builds skip them.

Use **Check for updates** in the tray to see the immediate outcome. You can also check the [release page](#) and follow the [installation guide](#). Turning off automatic checks does not disable the manual menu action.

Why did the installer show an operating-system warning?

Updater signatures and OS code signing are different mechanisms. An update can be signed for Sevak's updater while an installer is not yet signed for Windows or notarized by Apple. Read the [platform installation notes](#) for the current status and make sure the package came from the project's release page.

Does Sevak work offline?

App search, indexed file search, calculations and local ranking work without network access. Web searches, checking for a release, and downloading updates need an internet connection. Indexes depend on locally accessible applications and folders.

Where is my data? Does Sevak collect analytics?

No telemetry. Sevak does not collect analytics. All config, usage history, and logs stay on your machine.

Network activity:

- **Optional update checks** (default on): Check GitHub for a new version at startup and daily. Disable with `[general] check_for_updates = false`.
- **Web searches:** When you run a web search, your browser contacts that search provider (Google, YouTube, GitHub, etc.). Sevak does not proxy or log these.
- **Currency conversion** (off by default): When enabled, Sevak downloads the European Central Bank's daily rates once per day for currency conversion.

See [Files and data](#) for where everything is stored, and [Configuration](#) for the settings.

Reporting a problem

Open an issue through the [repository's issue form](#) and include:

- Sevak version and operating system.
- Linux desktop and X11/Wayland session type, if relevant.
- The query or shortcut that reproduces the issue.
- What you expected and what happened.
- Relevant recent log lines, with private paths or unrelated data removed.

Send security issues through [SECURITY.md](#) instead of a public issue.

Frequently asked questions

Getting started

How do I change the hotkey?

Open **Settings** → **General**, then change **Shortcut** to a key combination not already in use by your desktop or another app. Click **Save**.

On **Linux Wayland**, click **Set up GNOME shortcut** to create the binding in your desktop.

More: [Custom hotkeys](#).

How do I make Sevak start at login?

Open **Settings** → **General** and turn on **Launch at login**. Click **Save**.

Why doesn't my search show any results?

1. Check that the plugin is **enabled** in **Settings** → **Plugins** (e.g., Applications, Files)
2. If searching files with **f**, make sure the folder is added in **Settings** → **Files** → **Folders to search**
3. Reload the index: click the tray icon and select **Reload index**

Why is a recent app or file missing?

After installing an app or adding files, choose **Reload index** from the tray menu to refresh Sevak's index. New results appear on the next search.

How do I search inside a folder?

Type the path: `~/Documents/` (note the trailing slash). Sevak lists the contents. Use ↑/↓ to select and **Tab** to drill down. **Shift+Tab** goes back up.

For ordinary searches to also show file results, turn on **Settings** → **Search** → **Show in global results**.

Universal Actions

What is Universal Actions?

Press **Ctrl + Alt + Space** (or your configured hotkey) to show what you can do with what you have selected: text, a link, or files in a file manager.

Sevak copies the selection, shows the available actions and puts your clipboard back. Read more: [Universal Actions](#).

Why doesn't Universal Actions work on Wayland?

Applications on Wayland cannot read each other's selections or send keystrokes. Workaround: enable **Use the clipboard if the selection can't be read** in **Settings** → **General**, copy the text yourself, then press  +  + .

Why doesn't Universal Actions work in terminal windows?

On Windows and Linux, Sevak skips terminal windows (because  +  would interrupt the running program). Select the text with your mouse and use the clipboard fallback instead.

Appearance and display

How do I use a dark theme?

Open **Settings** → **Appearance** and set **Theme** to **Dark**. Click **Save**.

Or set `[appearance] theme = "dark"` in `config.toml`.

Can I customize the colors?

Yes. Open **Settings** → **Appearance**:

- **Accent color**: set the highlight color (#rrggbb or rgb(...))
- **Custom stylesheet**: name a CSS file in your config folder to override any theme variable

More: [Themes and appearance](#).

The launcher is too small or too big. Can I resize it?

Open **Settings** → **Appearance** and adjust **Window width**. The height scales with the font size.

To change the font size, adjust **Font size** under Appearance (affects the whole launcher).

Files and searching

How do I search only files?

Type the keyword followed by a space and filename. Default keyword is `f: f project`.

Change or disable the keyword in **Settings** → **Files** → **Keyword**.

More: [File search](#).

Can Sevak search inside document text?

No, Sevak searches filenames and folder names only, not document contents. For full-text search, use your operating system's built-in search (Windows Search, Spotlight, etc.) or specialized tools.

The file index is incomplete or seems stuck.

1. Check in **Settings** → **Files** that the folder is listed
2. Check that **Folder depth** is enough for your nested folders
3. Choose **Reload index** from the tray menu
4. Wait for indexing to finish (check the launcher: it shows a status message if indexing is running)

If a folder is slow or unreachable, indexing continues without blocking.

Calculations and conversions

Why doesn't currency conversion work?

Currency conversion is **off by default** because it needs a network connection. Turn it on in **Settings** → **Plugins** under **Currency conversion**. Click **Save**.

On the first search, Sevak downloads exchange rates from the European Central Bank and caches them for 24 hours.

Can I use other currencies?

Sevak supports 30+ ISO currency codes and symbols. Use `100 usd in eur`, `50 € to $`, or `$100 in eur`. See [Currency conversion](#).

How do I calculate percentages?

The `%` operator is remainder, not percentage. For 15% of 200, type `200*15/100` or `200*0.15`.

More: [Calculator](#).

Customization and advanced

How do I add a keyboard shortcut that runs a command?

In **Settings** → **Hotkeys**, click **Add hotkey**:

- Set the key (e.g., `^ Ctrl` + `⌘ Alt` + `T`)
- Choose **Query** and type `>` to open the launcher with a shell command ready
- Or choose **Run** and type a result ID (e.g., `system:lock`)

More: [Custom hotkeys](#).

Can I add my own search engine?

Yes. Open **Settings** → **Web search**, click **Add engine** and fill in:

- **Keyword:** what you type (e.g., `so` for Stack Overflow)
- **Name:** display name
- **URL:** search URL with `{query}` placeholder (e.g., `https://stackoverflow.com/search?q={query}`)

Click **Save**.

More: [Web search keywords](#).

How do I write a script plugin?

Script plugins let you add custom keywords without rebuilding Sevak. Put a folder with a `plugin.toml` and a script in the `plugins` folder next to `config.toml`.

See [Script plugins](#) and the [Plugins guide](#). Three examples are in `examples/plugins/`.

Privacy and data

Does Sevak send my data anywhere?

No. Your searches, files, clipboard and settings stay on your machine. Sevak makes only two optional network requests:

1. **Updates** (off by default in config): checks GitHub once a day
2. **Currency rates** (off by default): downloads ECB daily rates once per day if you enable it

Nothing is uploaded. [Read the full privacy policy](#).

Where are my settings and data stored?

See [Files and data locations](#).

Can I sync my config to another machine?

Yes. Copy your config folder to another computer (or store it in a synced folder like Dropbox). Sevak reads it on startup.

See [Files and data locations](#) for where your config is stored.

Troubleshooting

The launcher won't open. What do I do?

1. Check that Sevak is running: look for it in your system processes
2. Try the CLI: open a terminal and run `sevak` or `sevak --toggle`
3. Restart Sevak: run `sevak --quit`, then open it again
4. Check the logs: **Settings** → **Open config file** folder, look for logs

More: [Troubleshooting guide](#).

I see an error message in Settings. What does it mean?

Validation errors appear under the field that has a problem (shown in red). Fix the value and try again. Common issues:

- **Hotkey already in use:** the key is registered by another app. Choose a different one.
- **Invalid path:** the file or folder does not exist or is not accessible. Check the path.
- **Invalid format:** the value does not match the required format (e.g., color must be `#rgb` or `#rrggbb`)

More: [Troubleshooting](#).

Sevak takes a long time to start. Can I speed it up?

1. Reduce the number of indexed folders in **Settings** → **Files** → **Folders to search**
2. Lower **Folder depth** to skip deep nested directories
3. Disable plugins you don't use in **Settings** → **Plugins**
4. If indexing is slow, exclude large generated directories (e.g., `node_modules`, `.git`) — they are usually pruned automatically, but check your index

You can run Sevak in the background with `sevak --background` and press the hotkey to show it when you need it.

How do I reset Sevak?

Sevak reads `config.toml` at startup. To reset to defaults:

1. Close Sevak: `sevak --quit`
2. Open the config folder (**Settings** → **Open config file** folder)
3. Delete `config.toml`
4. Restart Sevak; a fresh config is created with defaults

Settings, usage history and clipboard history are in the same folder.

Wayland (Linux)

Why doesn't Alt+Space work on GNOME Wayland?

Wayland prevents apps from registering global hotkeys directly. Sevak asks you to run `sevak --setup-hotkey` to create a GNOME desktop shortcut instead.

In Settings, click **Set up GNOME shortcut** to do this automatically, or run the command from a terminal.

More: [Setting up the hotkey on Linux](#).

Can I use Sevak on other Wayland desktops (KDE, Weston, etc.)?

Partial support. On KDE and other desktops, you can:

- Use `sevak --toggle` from a terminal or custom keybind
- Leave Sevak running and use the tray icon

Global hotkeys are not possible without desktop-specific setup (if available). Universal Actions and pasting do not work on Wayland.

See [Installing on Linux](#).

macOS

How do I allow pasting and Universal Actions?

System Settings → **Privacy & Security** → **Accessibility**: add Sevak to the list. This permission lets Sevak paste results and read your selection for Universal Actions.

Why doesn't Cmd+Space work?

Cmd+Space is reserved by Spotlight. Choose another key in **Settings** → **General**.

Performance and limits

How many files can Sevak index?

The index is capped at **100,000 entries**. If you hit the limit, reduce the number of folders or lower the depth.

Is there a limit to how many snippets I can create?

No hard limit, but very large configs (100+ entries) may slow down loading slightly.

Why is search slow when I have thousands of files?

File search uses fuzzy matching on filenames, which is fast. If search is slow, check:

1. Are you searching a very large folder tree? Reduce **Folder depth** or add specific folders only
2. Is indexing still running? Look for a status message in the launcher
3. Are you on a slow or network drive? Local drives are faster

Questions?

Still stuck? Check the [Troubleshooting guide](#) or open an issue on [GitHub](#).

Privacy

Sevak is local-first. Your data stays on your machine except when you explicitly choose a network feature.

What stays local

- Application index, files and bookmarks
- Search history (if enabled)
- Clipboard history (if enabled)
- Snippet library
- Settings and configuration
- Usage statistics (how often you run each result)
- Logs (for debugging)
- Script plugins and their data

All data is stored in a single folder on your machine; none is uploaded anywhere.

What touches the network

Only when you opt in or use network features:

Web search

When you search with a web keyword or open a web search result, your query is sent to the selected search engine (Google, YouTube, GitHub, etc.) in your browser. Your search engine provider receives the query. This is **always your choice** — you type a web keyword and press Enter, or click a web result.

Example: typing `g rust traits` constructs `https://www.google.com/search?q=rust+traits` and opens it in your browser. Google sees `rust traits` as your query.

Currency conversion

Opt-in feature (off by default). When enabled via `[calculator] currency = true`:

- Sevak downloads the **European Central Bank's daily reference rates** once per day
- URL: `https://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml` (small XML file, ~1.5 KB)
- No account or API key required
- Downloaded in the background (not during typing)
- Cached locally and reused for 24 hours

- Kept in `currency-rates.json` in your data folder

The ECB does not require authentication. Your IP address and request timestamp are visible to the ECB's servers, as with any web request. Sevak does not send additional context.

Disable with `[calculator] currency = false` to turn off the network request.

Update checks

Opt-in feature (on by default). When enabled via `[general] check_for_updates = true`:

- Sevak checks **GitHub Releases** at startup and once per day for a new version
- URL: `https://github.com/ninad-k/Sevak/releases/latest/download/latest.json` (small JSON metadata)
- No account required
- Done in the background (not during typing)
- Checking only; updates are never installed without your permission

Disable with `[general] check_for_updates = false` to turn off the network request.

What is NOT collected

- No telemetry (no tracking, analytics, or usage data sent anywhere)
- No crash reports
- No advertising
- No profiling or behavioral analysis
- No timestamps of your searches or results
- No identifiers or user profiles

Script plugins run with your permissions and can make network requests themselves; review what they do before installing them.

Local data files

Inside your config folder (see [Files and data locations](#)):

File	Purpose	Sent elsewhere?
<code>config.toml</code>	Your settings	No

File	Purpose	Sent elsewhere?
<code>usage.json</code>	Frequency/recency of results; search history (if enabled)	No
<code>currency-rates.json</code>	Cached ECB rates (if currency enabled)	No
<code>plugins/</code> folder	Script plugins and their data	No, unless the plugin makes network requests
Logs	Diagnostic output for troubleshooting	No (you can share them manually)

Clipboard behavior

- **Pasting:** when you paste a result, Sevak hides, brings the previous window back, and presses `^ Ctrl + V` (`⌘ Cmd + V` on macOS). The app itself receives the text.
- **Copying:** `^ Ctrl + C` in the launcher copies the selected result's value or path to your clipboard. Your OS clipboard history (Windows Win+V, macOS, or a clipboard manager) may record it.
- **Accessibility:** on macOS, pasting and Universal Actions need the **Accessibility** permission. You grant this once in System Settings.

Selection reading (Universal Actions)

When you press the Universal Actions hotkey:

1. Sevak reads your system clipboard (and on X11 Linux, your PRIMARY selection)
2. Presses `^ Ctrl + C` (`⌘ Cmd + C` on macOS) in the app you were using
3. Waits up to 0.3 seconds for the copy to complete
4. Reads the new clipboard content (text or files)
5. Restores your previous clipboard

The original app makes the copy, not Sevak. Your OS clipboard history may see it. Sevak's own clipboard history does not record this copy. The selection is never stored.

On **Windows and macOS**, this works everywhere except admin windows (Windows only). On **Linux Wayland**, app-to-app selection reading is not possible; use the fallback: copy the text yourself and set `[actions]`

```
use_clipboard_fallback = true.
```

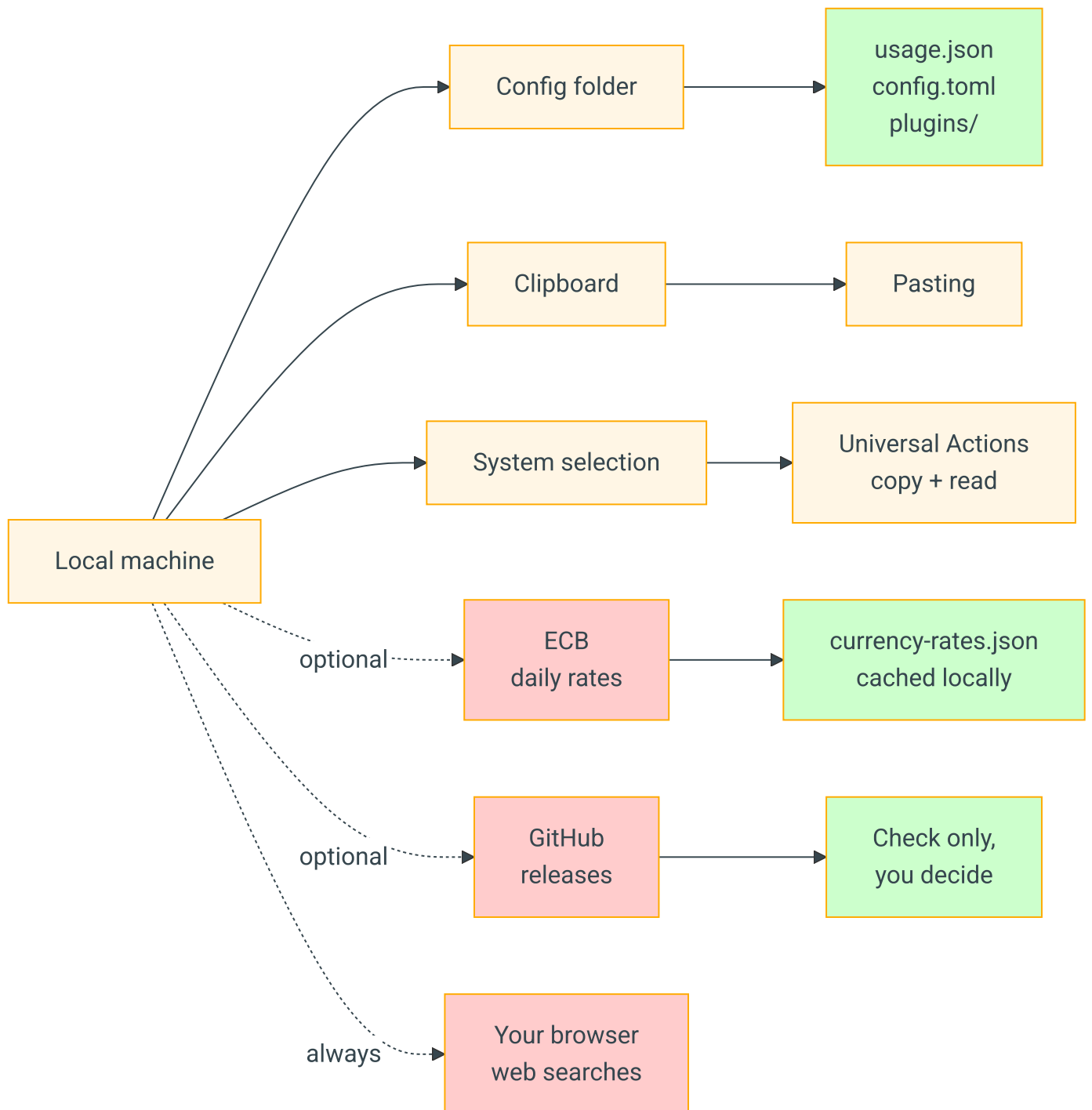
Data locations

See [Files and data locations](#) for where everything is stored.

Philosophy

Sevak is designed to be helpful without demanding trust. No cloud, no accounts, no surprises. You can verify what it does by reading the source code on GitHub.

Architecture diagram



Red borders (top-right) = network. Green borders (bottom) = stays local.

Developers